

1. Data Science – Machine Learning – Introduction

Contents

1. Machine learning.....	2
2. Is machine learning hard?	2
3. Program	2
4. What is an algorithm?	2
5. What is machine learning algorithm?.....	3
6. Diff b/n machine learning algorithm and normal algorithm?	3
7. Why Machine learning?	3
8. Machine learning definitions	4
9. Artificial intelligence vs Machine Learning vs Deep Learning.....	5
9.1. Machine learning	6
9.2. Deep learning	7
9.3. Artificial intelligence	8
10. Is computer identifying pictures on image like cat/dog?	9
11. Is human identifying picture on image like cat/dog?	9
12. Human vs computer	10
13. How a machine can take the decisions?.....	11
14. Past couple of decades.....	11
15. Human's follows this formula.....	11
16. Generally.....	12
17. How do machines think?.....	13
18. Real time examples	14
19. Gmail application	14
20. Banking loan application / credit card	15
21. Where machine learning helps?	16
22. Few of machine learning applications	16
23. Why machine learning required?	16
24. Technically speaking.....	17

1. Data Science – Machine Learning – Introduction

1. Machine learning

- ✓ It is a technique which enables computers to learn automatically from past data.
- ✓ It is an approach to train the models.
- ✓ It is helpful to predict the future values.

2. Is machine learning hard?

- ✓ No, never
- ✓ Machine learning is very easy.
- ✓ It requires imagination, creativity, and a visual mind.
- ✓ Key point is, we should learn how to play with data and applying logic on data.
- ✓ This material helps how to do all these things.

3. Program

- ✓ A program is a group of instructions to perform the task; computer can execute these instructions to finish the task.
- ✓ This is very good approach for simple task

4. What is an algorithm?

- ✓ An algorithm is program which having group of instructions to perform a task.
 - Input + Logic = Output
 - Data + Program = Result

5. What is machine learning algorithm?

- ✓ Machine Learning algorithm is an application.
- ✓ It **learns** knowledge (patterns) automatically **from the data** without being explicitly programmed.
 - $\text{Data} + \text{Result} = \text{Program}(\text{Model})$

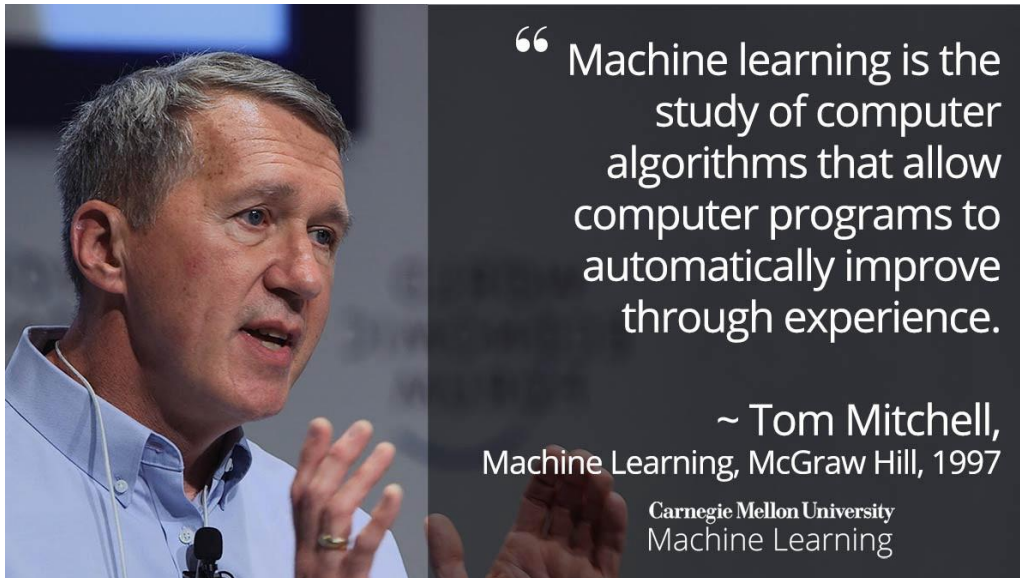
6. Diff b/n machine learning algorithm and normal algorithm?

- ✓ Machine Learning algorithm learns knowledge (patterns) from the Data.
- ✓ Normal algorithm cannot learn anything on its own.

7. Why Machine learning?

- ✓ Currently every company is generating huge amount of the data.
- ✓ So, the volume of data is increasing on every day.
- ✓ Machine learning algorithms can extract information from data.
- ✓ The major use cases of Machine learning,
 - Creating models.
 - Getting deep insights & etc.
- ✓ Machine learning is going to be the center point of all fields.

8. Machine learning definitions



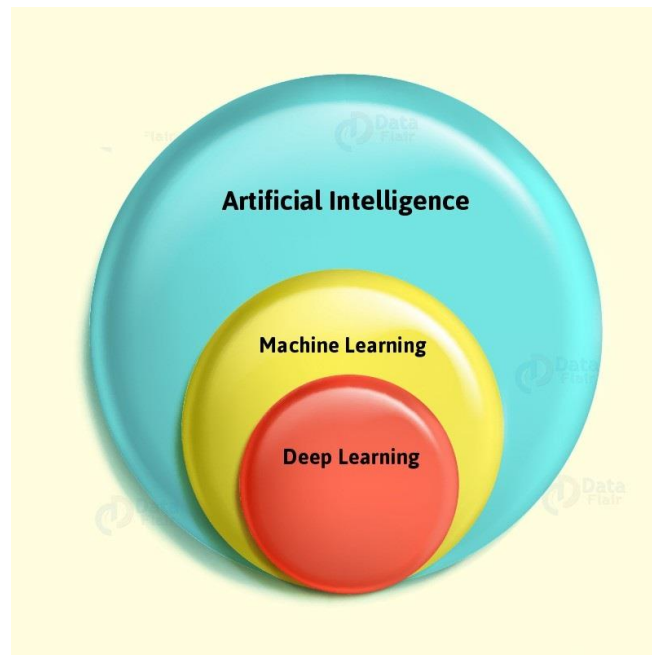
Arthur Samuel definition

- ✓ Machine learning term was introduced by Arthur Samuel in the year of 1959 and he defined in this way,
 - Machine learning is,
 - Enables a machine to learn automatically from data,
 - Improve performance from experiences,
 - Predict things without being explicitly programmed

Daniel's definition

- ✓ Teaching to computers how to learn by using data and experience, rather than by instructions.

9. Artificial intelligence vs Machine Learning vs Deep Learning



9.1. Machine learning

- ✓ Machine learning is a part of artificial intelligence
- ✓ Machine Learning is a technique of understand the data, learn from data and then apply what they have learnt to take next decision.

Examples

- ✓ Amazon using machine learning to give better product choice recommendations to their costumers based on their preferences.
- ✓ Netflix uses machine learning to give better suggestions to their users of the TV series or movie or shows that they would like to watch & many more

9.2. Deep learning

- ✓ Deep learning is actually a subset of machine learning.
- ✓ The main difference between deep and machine learning is, machine learning models works better but the model still needs some guidance.
- ✓ If a machine learning model returns an inaccurate prediction then the programmer needs to fix that problem explicitly.
- ✓ In the case of deep learning, the model does it by itself.

Example

- ✓ Automatic car driving system is a good example of deep learning.

9.3. Artificial intelligence

- ✓ AI is an ability of computer program to function like a human brain.
- ✓ Machine learning and deep learning are the subsets of AI
- ✓ The MOTO of AI is to replicate a human brain, the way a human brain thinks, works and functions.
- ✓ Currently AI is not yet fully implemented but we are very close to establish that too.

Example

- ✓ Sophia, the most advanced AI model present today.

10. Is computer identifying pictures on image like cat/dog?

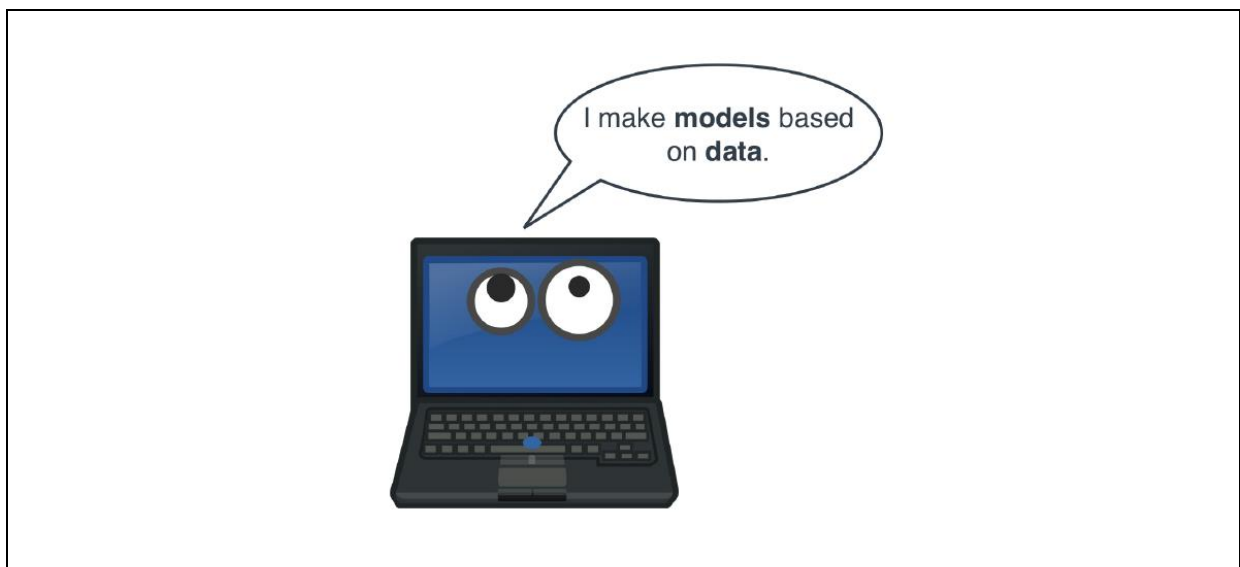
- ✓ Yes and it is really amazing task.
- ✓ If we giving lot of images to computer then computer can try to learn about the attributes to recognize images
- ✓ This is called as machine learning.
 - It is something like teaching to computer how to think like a human

11. Is human identifying picture on image like cat/dog?

- ✓ Absolutely **YES** right.
- ✓ During my childhood parents/teachers taught like how to recognize a cat/dog/tiger/lion/etc
- ✓ So, in real life, if we see cat/dog/tiger/lion (sorry tiger/lion I have seen in Zoo but not directly) then we can easy to recognize without much effort.
- ✓ So, a human can take the decision based on experience.

12. Human vs computer

- ✓ Humans take the **decisions** based on the **experience**
- ✓ Computer can create **models** by using data



13. How a machine can take the decisions?

- ✓ First we need to understand human decision making process before understanding about how a computer takes the decision.

14. Past couple of decades

- ✓ From past two decades most of the companies are digitalized.

15. Human's follows this formula...

- ✓ All humans used to follow some patterns or formulas to take the next best action.

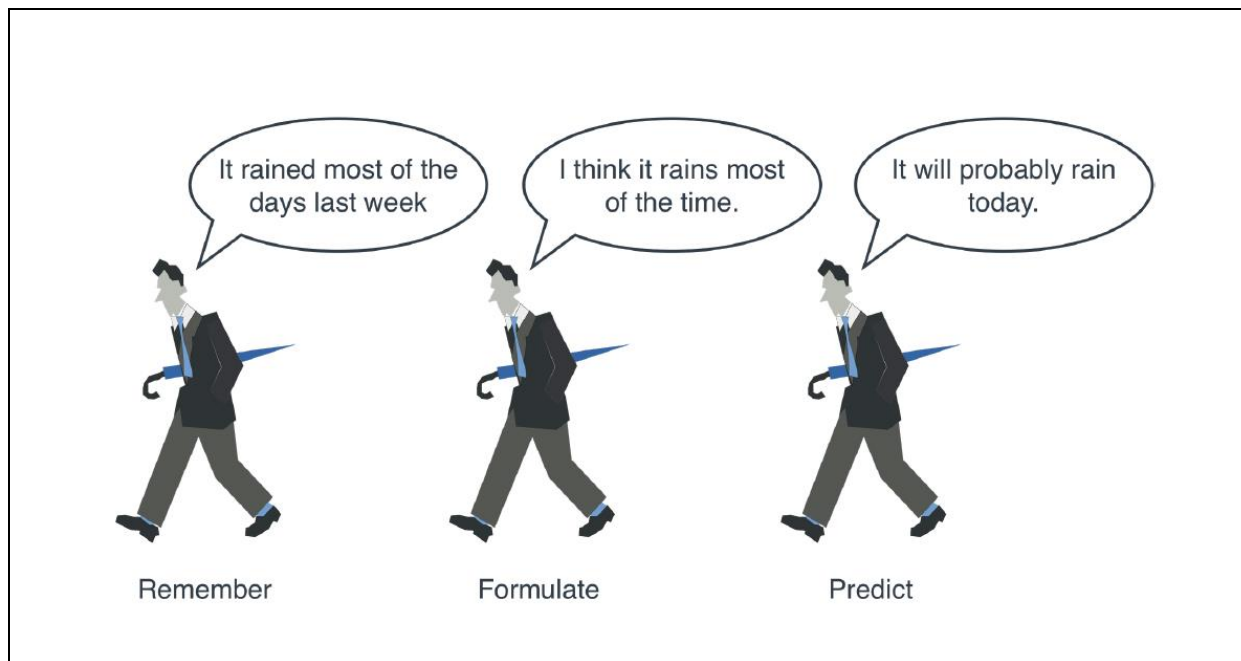
- Remember
- Formulate
- Predict

16. Generally

- ✓ We **remember** past situations that were similar.
- ✓ We **formulate** a general rule.
- ✓ We use this rule to **predict** what will happen if we take a certain decision.

Example

- ✓ For supposed to answer for a question like, "**Will it rain today? Or not**", then we will follow one **process** to answer this question right.
- ✓ We may be right or wrong, but at least, we are trying to make an accurate prediction.



17. How do machines think?

- ✓ The goal is to make the computer think the way we think, namely, how we use the remember-formulate-predict framework.
- ✓ Here are the points what the computer can understand the points

Steps

- ✓ **Remember**
 - Look at the given **HUGE** data.
- ✓ **Formulate**
 - Go through rules and formulas, and check which one fits for the data
- ✓ **Predict**
 - Use the rule to make predictions about future data.

18. Real time examples

- ✓ Gmail application
- ✓ Banking loan application or credit card eligibility & etc

19. Gmail application

- ✓ *Gmail* is the very well-known example.
- ✓ Hope we are all known about Spam folder too
- ✓ *Spam*
 - It is the common term used for junk or unwanted email, such as chain letters, promotions, and so on
- ✓ *Ham*
 - It is the common term used for non-spam mail means useful mail
- ✓ In this scenario machine learning algorithm is working to filter spam or not
- ✓ Machine learning algorithm,
 - Understand the previous mails(data)
 - Based on the previous mail creates some models.
 - These models applies on upcoming mails and predict whether it is spam or not

20. Banking loan application / credit card

- ✓ Banking loan application, this is the very common application hope everyone knows
- ✓ Once we applied for loan by submitting required documents(data) like pay slip, past 6 months banking statement and etc
- ✓ Then banking guys will run one application over the given data.
- ✓ This is Machine learning algorithm
- ✓ This algorithm passes through the data and predicts the weather the loan/credit card sanctioned or not.

Note

- ✓ Many other applications are using machine learning
- ✓ I would like to say, Machine learning is everywhere 😊

21. Where machine learning helps?

- ✓ From past two decades most of the companies are digitalized.
- ✓ So, here data is generated more and more.
- ✓ So, in this case machine learning helps us to develop predictive models and automate several things.
- ✓ Assuming that we had past couple of year's **amazon** transactions data.
- ✓ Some questions:
 - Wanted to know how the business was in last couple of years
 - Wanted to take best decisions to improve the business and etc
- ✓ To answer above questions, we need to,
 - Gather the data
 - Process the data
 - Take the decisions.

22. Few of machine learning applications

- ✓ Text processing
- ✓ Speech Recognition
- ✓ Traffic prediction
- ✓ Product recommendations
- ✓ Self-driving cars
- ✓ Email spam and malware filtering
- ✓ Online fraud detection
- ✓ Weather forecasting and prediction & etc

23. Why machine learning required?

- ✓ As a human we cannot access huge amount of data manually to process.
- ✓ We can train machine learning algorithms by providing huge amount of data.
- ✓ Machine learning algorithm travel through this data in order to learn about data, it create the model and predict results automatically for new data

24. Technically speaking...

- ✓ Technically speaking,
 - WITH THE help of historical data
 - MACHINE LEARNING algorithms
 - BUILD a mathematical MODEL
 - that helps in making PREDICTIONS
 - Without being EXPLICITLY PROGRAMMER

2. Data Science – Machine Learning – Terminology

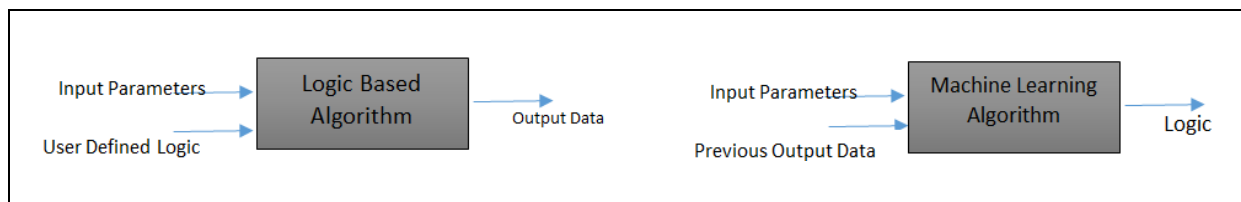
Contents

1. Machine learning.....	2
2. Model.....	2
3. Train	3
4. How to train?.....	3
5. Model learns.....	3
6. Pattern	3
7. Before trained	3
8. After trained	3
9. Test the model.....	3
10. Model deployment.....	4
11. Towards target value or classifier.....	4
12. General example	4
13. Smart application	4
14. Normal application.....	5
15. Smart application	6
16. Use case	6
17. Accuracy testing	7

2. Data Science – Machine Learning – Terminology

1. Machine learning

- ✓ Machine learning is a technique which enables computers to learn automatically from past data.
- ✓ Machine Learning is an approach to train the **models**.
- ✓ Machine learning is helpful to predict the future values.



2. Model

- ✓ In simple terms, a model is a representation of reality.
- ✓ A model is created to do a specific purpose

Example

- ✓ General example, a world map is a model for this physical world or universe.
 - The creator of world map have done deep analysis, understands, visualized with simple representation.

In terms of Machine learning,

- ✓ A model can be a,
 - A piece of code or program
 - A mathematical formula
 - A program + mathematical formula

3. Train

- ✓ We need to **train** the model

4. How to train?

- ✓ We need to use **data** to train the model

5. Model learns

- ✓ Model will learn **pattern** from data.

6. Pattern

- ✓ Patterns means a simple knowledge gained during training.
- ✓ So, model learnt patterns from data during training

7. Before trained

- ✓ Before training model, model doesn't know about the patterns.

8. After trained

- ✓ After training model, model knows about the patterns.

9. Test the model

- ✓ Once after model got trained then that model has to be tested to **check the accuracy**.
- ✓ If accuracy is good than the model works well

10. Model deployment

- ✓ Finally we need to deploy knowledge acquired by the model
- ✓ These models also called as model fit

11. Towards target value or classifier

- ✓ Models learnt patterns from data towards target value.
- ✓ So, let's try to understand what it means.

12. General example

- ✓ Assuming that one of my friend is introducing his friend as
 - His name is Prasad working in X Company and having 3 years of experience.
- ✓ When I heard about his experience I can guess/predict his salary (target value).
- ✓ So, our brain is already trained to guess some attributes from past data (old friends)
- ✓ So, we have patterns towards target value

13. Smart application

- ✓ Once accuracy satisfied then we need to deploy the model.
- ✓ Once model deployed then that application will become as smart application

14. Normal application

- ✓ Assuming that we have created an application by using Java/dotnet/Python.
- ✓ It's just called as normal software application because this application is just works on instruction based.
- ✓ Instruct based means,
 - If x value is greater than y value, then prints x value
 - If x value is less than y value, then prints y value

Make a note

- ✓ Normal application cannot think like human to take decisions
- ✓ Normal applications cannot take own decisions

15. Smart application

- ✓ A smart application is an application which thinks like a human to take the decisions.
- ✓ Smart application is intelligence based.

16. Use case

- ✓ Whenever you send email automatically Gmail application is predicting whether it is a spam or not spam.
- ✓ That means Gmail application is well trained and captures the patterns about mail type like,
 - Spam mail
 - Promotion mail
 - Social media mail & etc
- ✓ So, Gmail application knows patterns about to recognize the type of mail.

17. Accuracy testing

- ✓ Before deploying the model, it needs to be tested in lot of dimensions.
- ✓ If model is getting good predictions then we need to deploy.

Process in machine learning

- ✓ We need to **train** the model.
- ✓ During training model acquires the **knowledge**.
- ✓ This knowledge is called as **model** fit
- ✓ Once after model fit produced we need to **test** to check the **accuracy**.
- ✓ If satisfied with accuracy then we need to **deploy** the model

Data is important

- ✓ We need to train the model with data
- ✓ During this training model will go through the data and understand the patterns about data.
- ✓ These patterns helps to predict the result on **new data**

3. Data Science – Machine Learning – Data & ML Algorithm

Contents

1. Data.....	2
2. Data in table	3
3. Row	3
4. Column	3
5. Cell.....	3
6. Statistical Learning Perspective	4
6.1. Example.....	4
7. Why Machine learning algorithms learn this function?.....	6
8. Terminology in statistics,	7
9. Computer Science Perspective.....	8
10. Models and Algorithms	9

3. Data Science – Machine Learning – Data & ML Algorithm

1. Data

- ✓ It is a collection of facts.
- ✓ Facts mean,
 - Alphabets
 - Numbers
 - Alphanumeric
 - Symbol

◇	A	B	C	D
1		Column 1	Column 2	Column 3
2	Row 1	2.2	2.3	1
3	Row 2	2.3	2.6	0
4	Row 3	2.1	2	1
5				

2. Data in table

- ✓ Generally, tables having data like rows, columns and cells
- ✓ Tabular data is very easy to understand.

3. Row

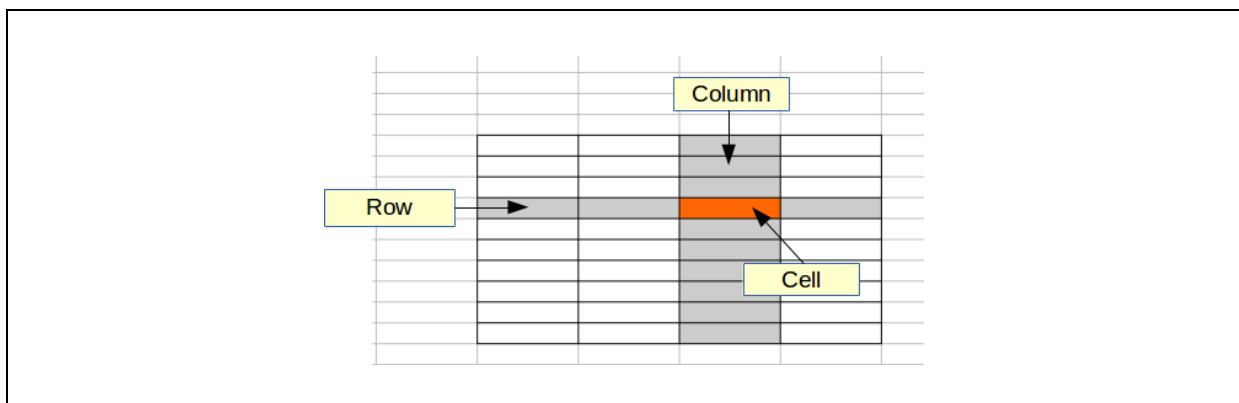
- ✓ A row describes a single entity or observation.
- ✓ A row is also called a record.

4. Column

- ✓ A column is a vertical group of cells within a table.
- ✓ A column having same type of values.
- ✓ In column we can store the data like weights, heights and prices etc info.

5. Cell

- ✓ A cell is a single value in a row and column.
- ✓ It may be a real value (1.5) an integer (2) or a category (red).



6. Statistical Learning Perspective

- ✓ In statistical learning perspective data means,
 - It is the context of a **hypothetical function** (f) that the machine learning algorithm is **trying to learn**.

6.1. Example

- ✓ Assuming that, there is a farmer in village and initially he had 1 acre land.
- ✓ While forming rice it get yields as below,

Area	Rice packets
1	10
2	20
3	30

If former having 4 acres land then how many rice packets it yields?

Answer

- ✓ A non-technical guy also can answer as, we will get **40** packets for **4** acres of land.

Area	Rice packets
1	10
2	20
3	30
4	?

Simple calculation

✓ Formula is,

- | | | | | |
|----------|----|---------------|---|-----------|
| ○ 1 acre | => | 1 x 10 | = | 10 |
| ○ 2 acre | => | 2 x 10 | = | 20 |
| ○ 3 acre | => | 3 x 10 | = | 30 |
| ○ 4 acre | => | 4 x 10 | = | 40 |

- ✓ Simple Formula is,
 - $\text{Rice_yield} = \text{land_size} \times 10$

7. Why Machine learning algorithms learn this function?

- ✓ To predict output for the given input.

$$\text{Output} = f(\text{Input})$$

- ✓ Columns are called as input variables.
- ✓ Predicted result is called as output variable or response variable.

$$\text{Output Variable} = f(\text{Input Variables})$$

◇	A	B	C
1	X1	X2	Y
2	2.2	2.3	1
3	2.3	2.6	0
4	2.1	2	1
5			

- ✓ Input data may have more than one input variable.
- ✓ The group of input variables are called as input vector.

$$\text{Output Variable} = f(\text{Input Vector})$$

8. Terminology in statistics,

- ✓ Input variables are called as Independent variables.
- ✓ Output variable are called as the Dependent variable.
- ✓ Here the output is dependent on a function of input.

$$\text{Dependent Variable} = f(\text{Independent Variables})$$

- ✓ Input variable representing with **X**
- ✓ Output variable representing with **y**

$$Y = f(X)$$

- ✓ If we have multiple input variables then it's represent as input vector x_1, x_2, x_3 for the data

9. Computer Science Perspective

- ✓ A row means it's an entity/observation/instance/object in a table.
- ✓ A Column is called as an attribute.
- ✓ During modeling and predictions,
 - Input is called as input attribute.
 - Output is called as output attributes.

Output Attribute = Program(Input Attributes)

◇	A	B	C	D
1		Attribute 1	Attribute 2	Output Attribute
2	Instance 1	2.2	2.3	1
3	Instance 2	2.3	2.6	0
4	Instance 3	2.1	2	1
5				

Output = Program(Input Features)

Prediction output = Program(Instance)

10. Models and Algorithms

- ✓ A model is a specific representation learned from data and the algorithm as the process of learning it.

$$\text{Model} = \text{Algorithm}(\text{Data})$$

4. Data Science – Machine Learning – Learning Function

Contents

1. Learning a Function	2
2. Purpose of learning-function	2
3. Machine learning algorithms	3

4. Data Science – Machine Learning – Learning Function

1. Learning a Function

- ✓ Machine learning algorithms are described as,
 - Learning a target function (f) which maps input variables (X) to output variable (Y)

$$\text{Output} = f(\text{Input})$$

$$Y = f(X)$$

- ✓ This function also having some error.
- ✓ This error is independent to the input data

$$Y = f(X) + \text{error}$$

2. Purpose of learning-function

- ✓ The purpose of this learning-function is to make predictions.
- ✓ The function which is having less error, that function will make the accurate predictions.
- ✓ This is called predictive modeling.

3. Machine learning algorithms

- ✓ Machine learning algorithms are techniques for estimating the target function (f).
- ✓ This function helps to predict the **output variable (Y)** for the given **input variables (X)**.
- ✓ Different machine learning algorithms having different assumptions about the form of the function, such as whether it is linear or nonlinear.

5. Data Science – Machine Learning – Types of the Models

Contents

1. Feature and label	2
1.1. Features	2
1.2. Label	3
2. Label Example	3
3. Labelled and unlabeled data	4
3.1. Labelled Data:	4
3.2. Unlabeled Data	5
4. Supervised learning definition	6
5. Unsupervised learning definition	7
6. Supervised learning example	8
7. Remember – Formulate - Predict.	9
8. Types of Supervised learning models	11
8.1. Regression models	12
8.2. Classification models	12
9. Unsupervised learning example	13
10. Types of unsupervised learning models	14
10.1. Clustering	15
10.2. Dimensionality reduction	16

5. Data Science – Machine Learning – Types of the Models

- ✓ In Machine learning there are mainly 3 types of models exists,
 - ✓ Supervised learning
 - ✓ Unsupervised learning
 - ✓ Reinforcement learning

Best tip

- ✓ Before understanding the types of algorithms let's try to understand terminology

1. Feature and label

1.1. Features

- ✓ Features are simply the columns of the table
- ✓ These features describes the about the data
- ✓ In the given data, Age, Gender, Experience and Salary are features

Age	Gender	Experience	Salary
20	M	4	40000
24	F	5	50000

1.2. Label

- ✓ The output we will get after training the model is called as a **Label**
- ✓ Requirement
 - Suppose I wanted to predict the salary who had 6 years of experience
 - We prepared a model and that model predicted the salary.
 - Here salary is called as a **label**
- ✓ Simple
 - We are trying to predict a feature based on the others, that feature is the label.

2. Label Example

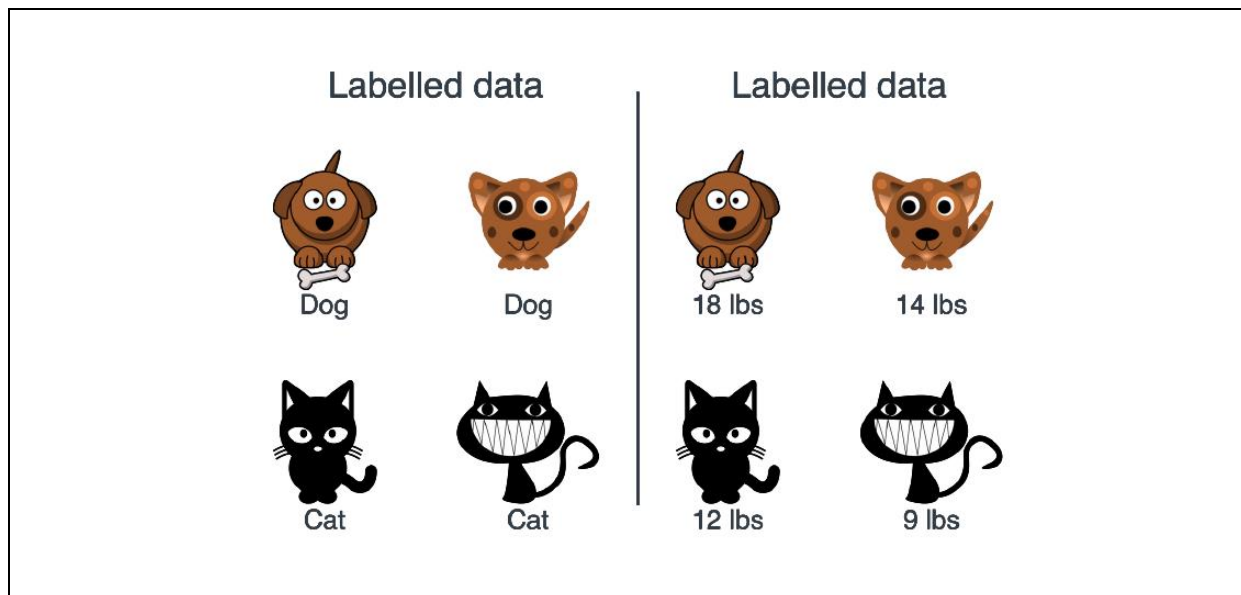
- ✓ If we are trying to predict the **type of pet** for example cat or dog based on information then that is the **label**.
- ✓ If we are trying to predict if the pet is **sick or healthy** based on symptoms and other information, then that is the **label**.
- ✓ If we are trying to predict the **age of the pet**, then the age is the **label**.

3. Labelled and unlabeled data

- ✓ According to the label, data is divided into two types
 - Labelled data
 - Unlabeled data

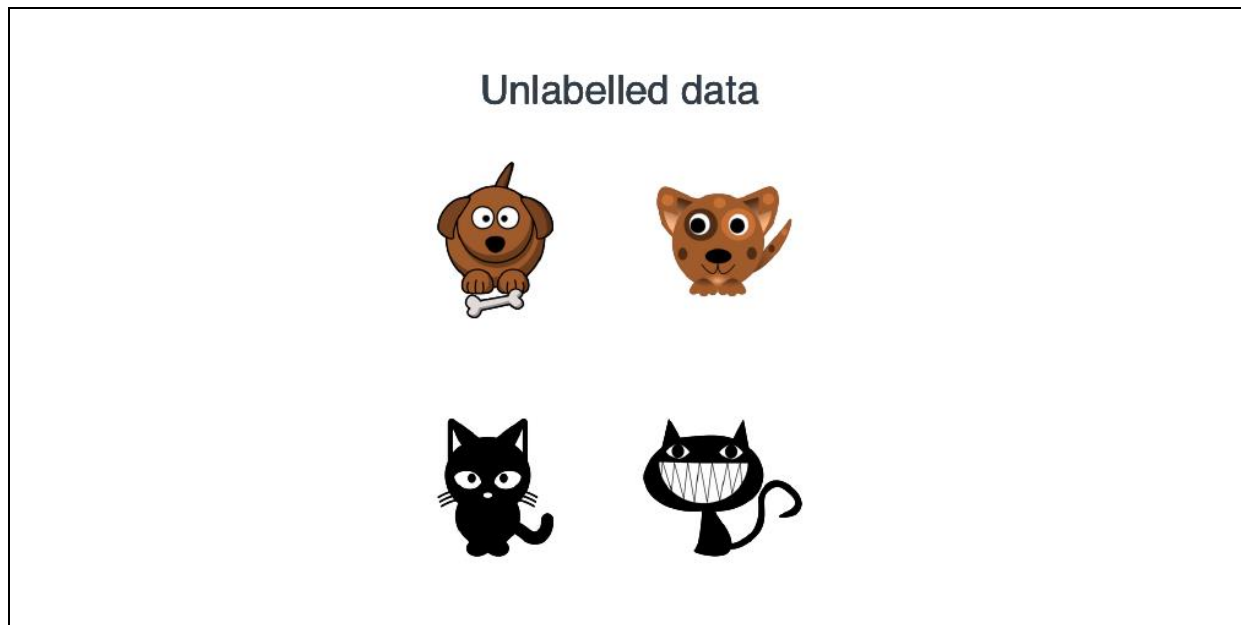
3.1. Labelled Data:

- ✓ Labelled data comes with a tag or label, like a name, a type, or a number.



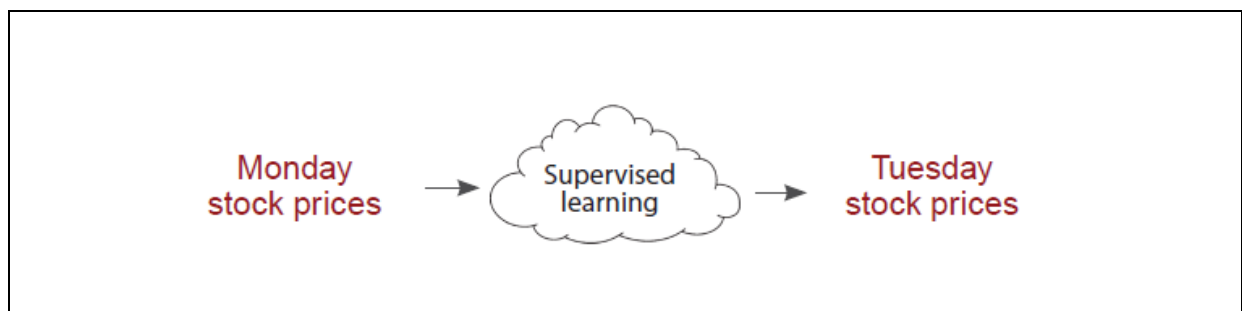
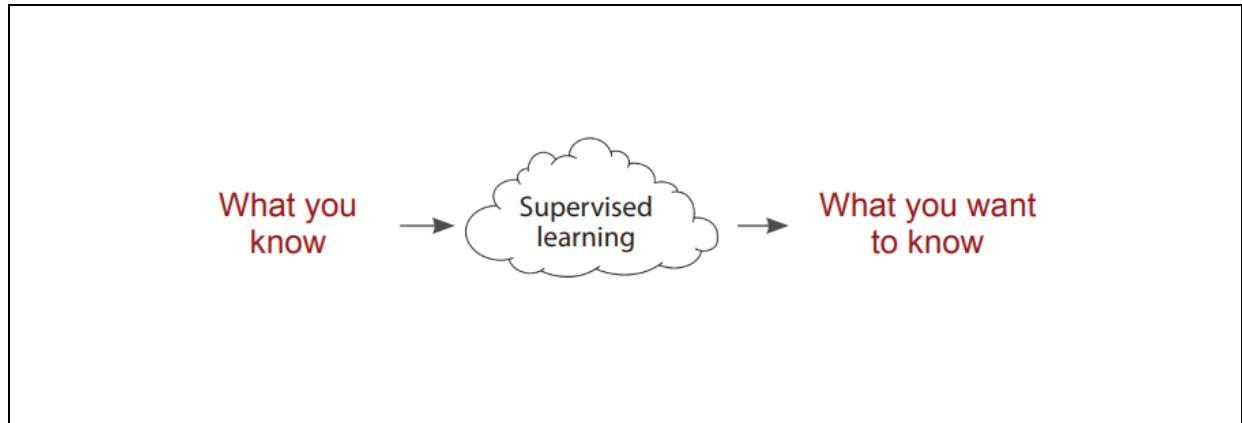
3.2. Unlabeled Data

- ✓ Unlabeled data have no tag or label



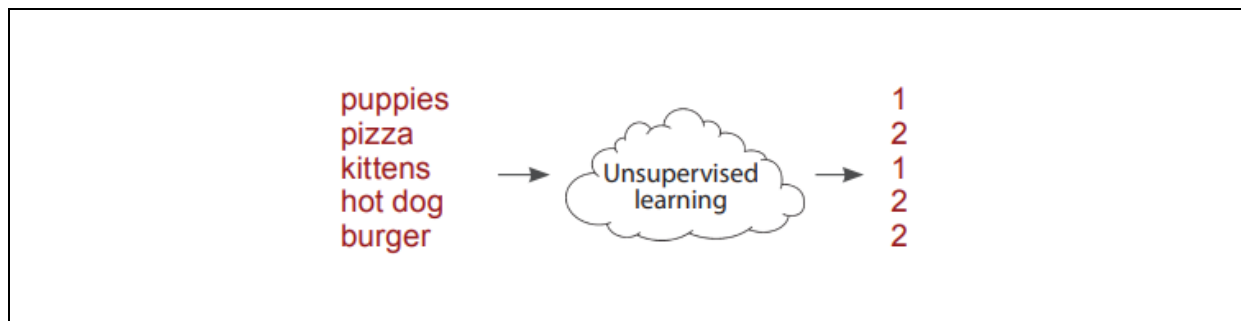
4. Supervised learning definition

- ✓ In supervised learning, we will train the models by using input **features** and **labels**



5. Unsupervised learning definition

- ✓ In unsupervised learning, we will train the models by using only input features and there is not labels



Unsupervised machine learning

Unsupervised learning groups your data.

6. Supervised learning example

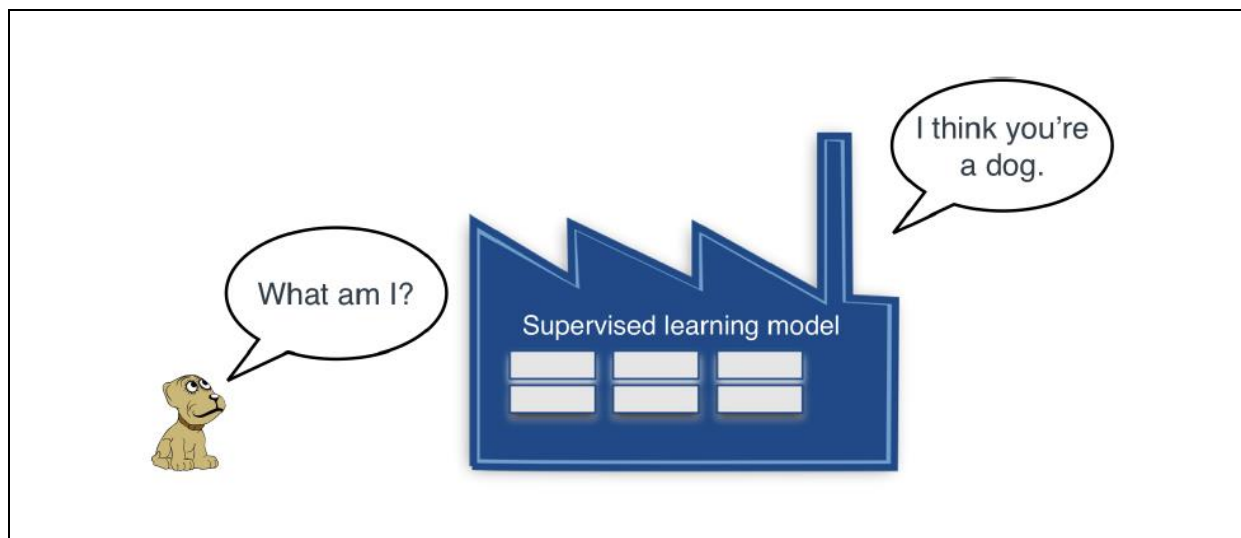
- ✓ Now a days it is very commonly using for all applications
- ✓ Supervised learning = **features** + **label**.

Examples

- ✓ Image recognition,
- ✓ Text processing,
- ✓ Recommendation systems & many more.

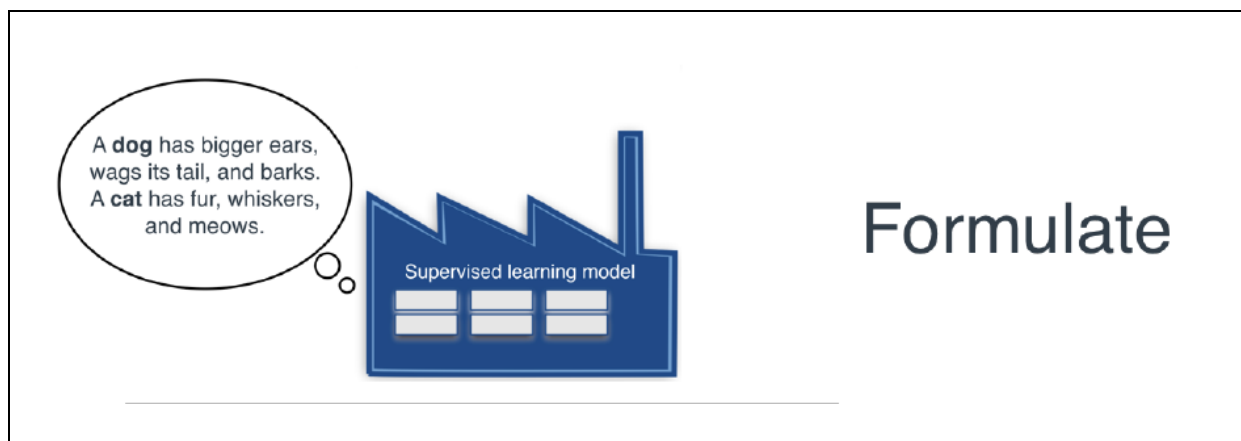
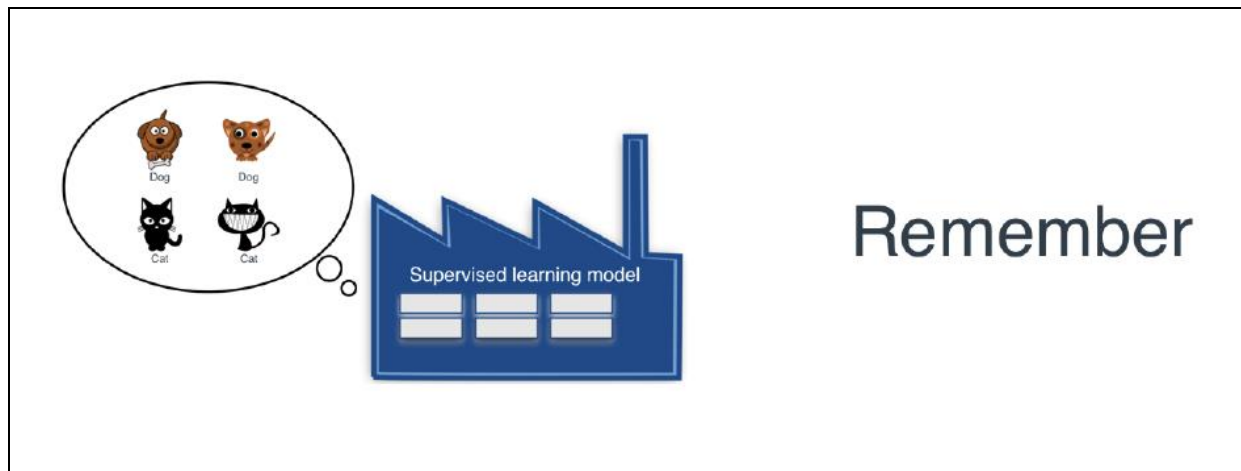
Scenario

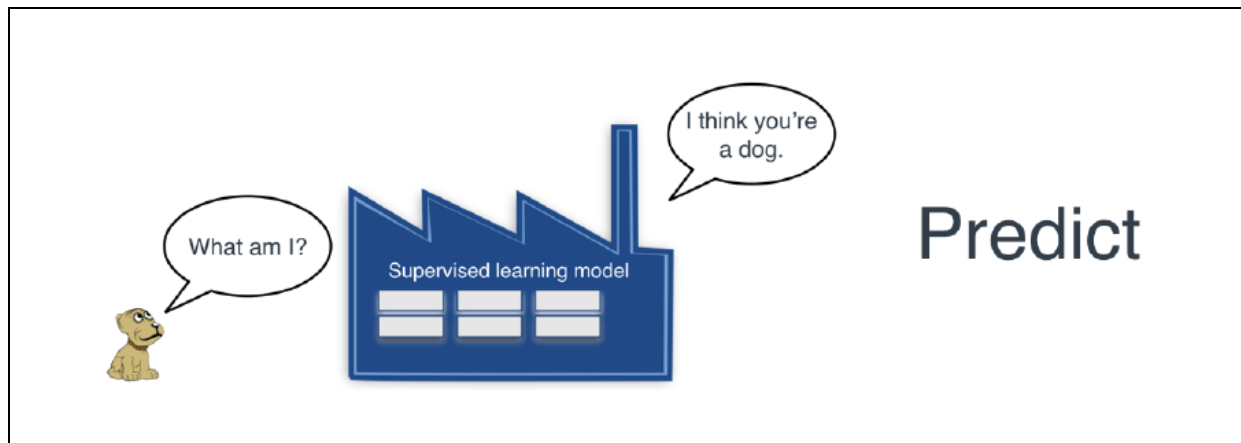
- ✓ In previous images, we have an images data about dogs and cats.
- ✓ **Labels** in the image are '**dog**' and '**cat**'.
- ✓ The machine learning model use previous data in order to predict the label of **new data points**.



7. Remember – Formulate - Predict.

- ✓ Supervised learning works in remember – formulate – predict
- ✓ The model first remembers the dataset of dogs and cats.
- ✓ Then model formulates a model for **what is a dog** and **what is a cat**.
- ✓ Whenever a **new image comes** in, the model makes a prediction about what the label of the image whether cat or dog





8. Types of Supervised learning models

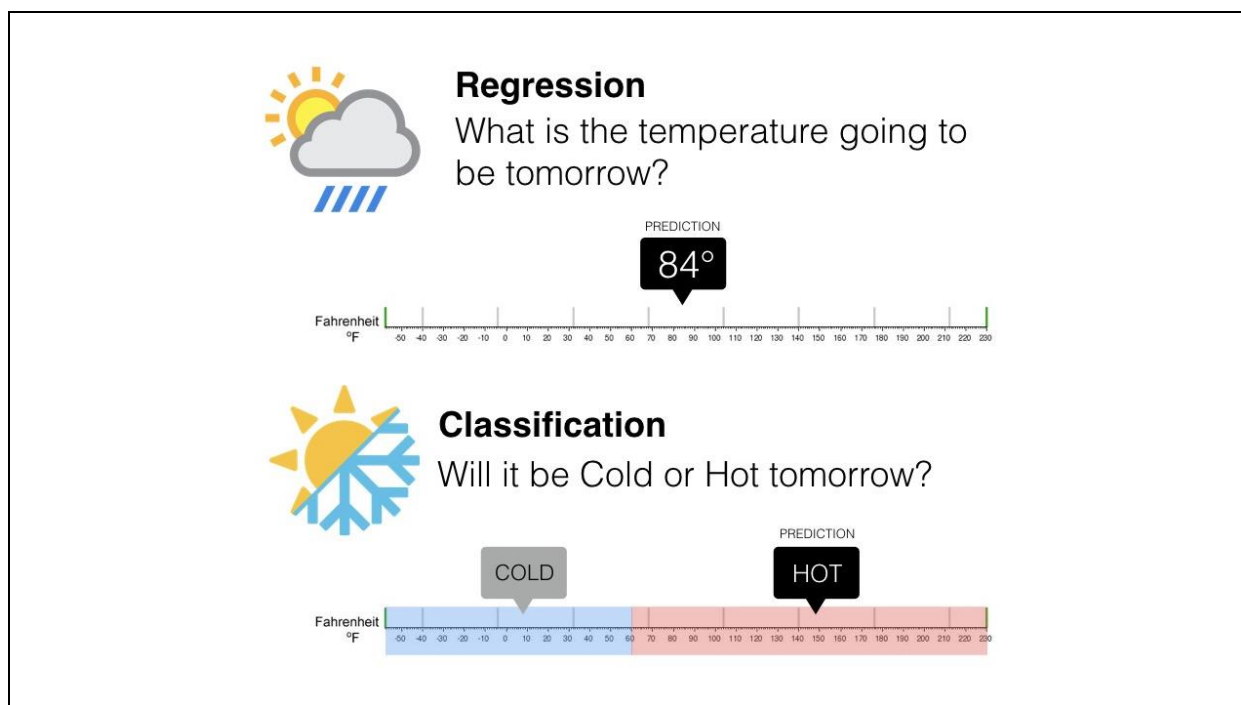
- ✓ Supervised learning models are divided into two types
 - Regression models
 - Classification models

8.1. Regression models

- ✓ Regression models used to predict a **number**
- ✓ The output of a regression model is a **continuous**, since the prediction can be any real value.
- ✓ Examples:
 - Weight of the animal
 - Employee salary
 - Students marks
 - Stock market
 - Number of sales
 - Predicting price of house & etc

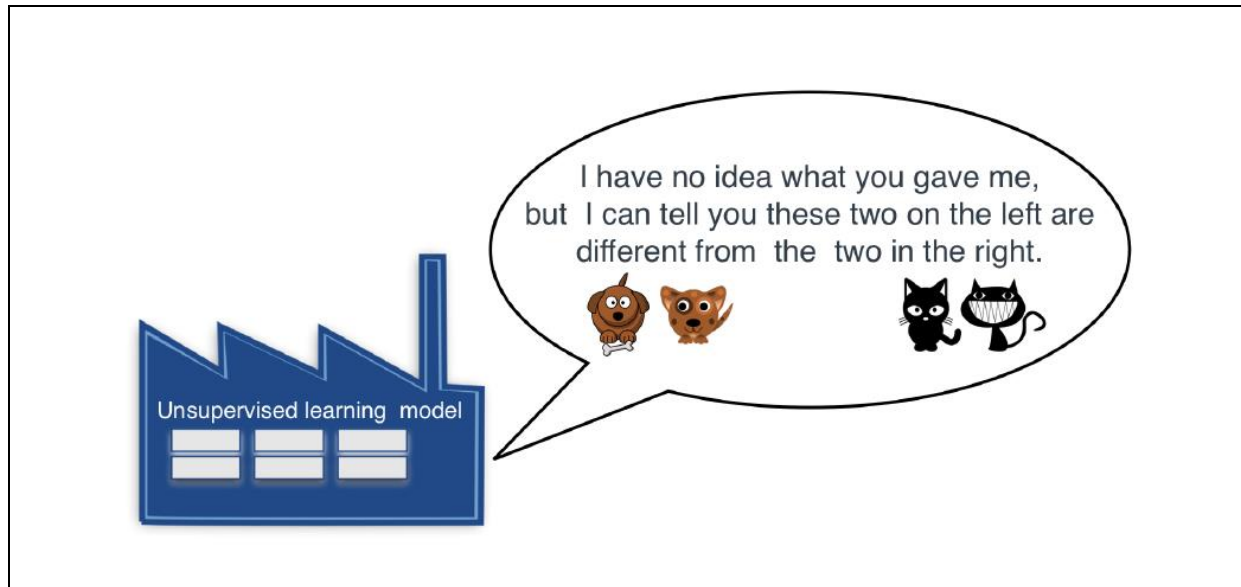
8.2. Classification models

- ✓ These are the types of models that predict the **class** or **state**.
- ✓ Examples
 - Type of animal (cat or dog),
 - Type of human being means male or female,
 - Biryani taste: good or bad or not good
 - Mail id spam or ham



9. Unsupervised learning example

- ✓ In unsupervised learning, we will train the models by using only input features but there are no labels
- ✓ Unsupervised learning is grouping the data based on similarities

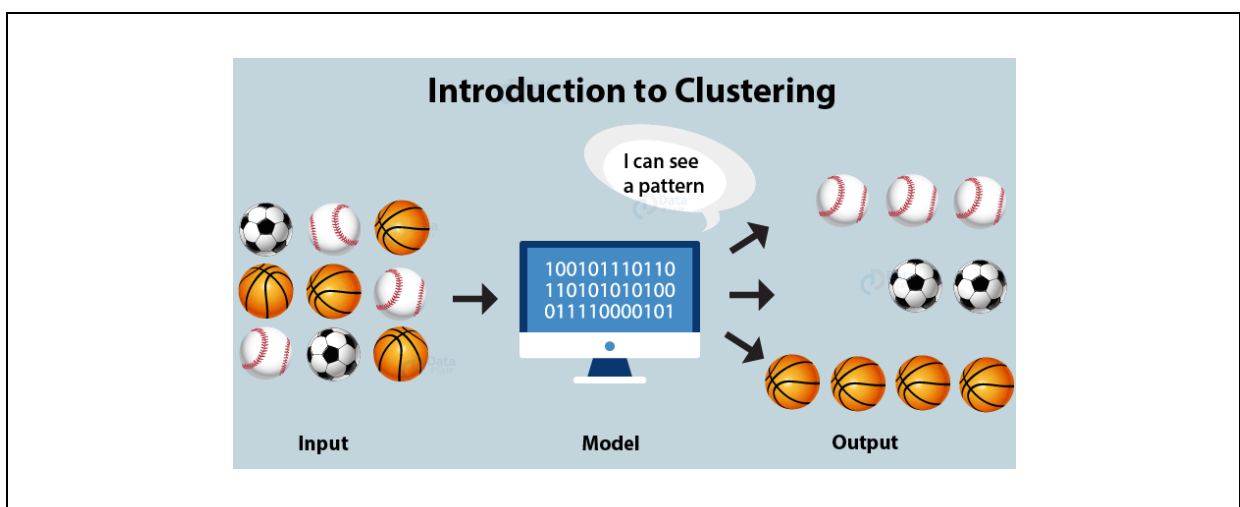
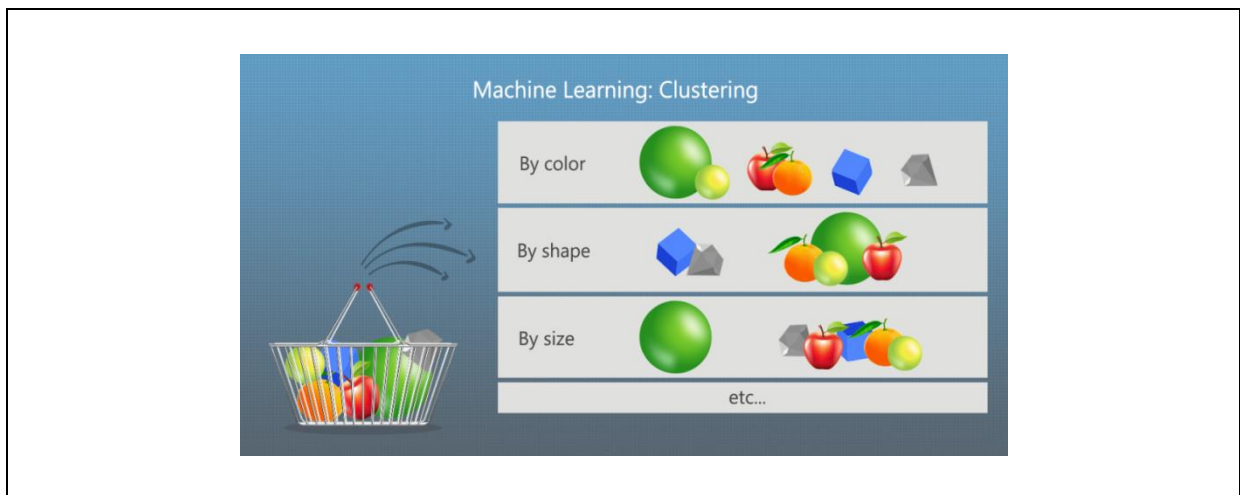
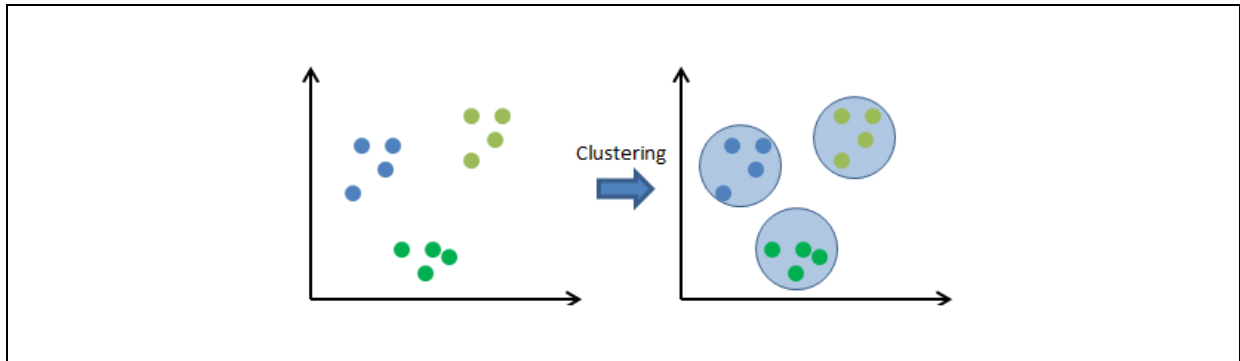


10. Types of unsupervised learning models

- ✓ Unsupervised learning models are divided into two types
 - Clustering
 - Dimensionality reduction

10.1. Clustering

- ✓ This is the task of grouping our data into clusters based on similarity.



10.2. Dimensionality reduction

- ✓ This is the task of simplifying our data and describing it with fewer features, without losing much generality.
- ✓ The dimensionality reduction algorithms will find ways that group them, losing as little information as possible.

6. Data Science – Machine Learning – Life Cycle

Contents

1. Machine learning life cycle	2
1.1. Data Collection.....	3
1.2. Data Preparation.....	3
1.3. Data Wrangling	3
1.4. Train the model.....	3
1.5. Test the model	4
1.6. Model deployment	4

6. Data Science – Machine Learning – Life Cycle

1. Machine learning life cycle

- ✓ There are mainly six steps involved here,
 - Data collection
 - Data preparation
 - Data Wrangling
 - Train the model
 - Test the model
 - Model deployment

1.1. Data Collection

- ✓ Data Gathering is the first step in machine learning life cycle.
- ✓ In this step, we need to identify the different data sources.
- ✓ Data can be collected from different sources such as files, database, web & etc.

1.2. Data Preparation

- ✓ During data preparation we should understand about the data format, quality of data & etc.
- ✓ A better understanding of data leads to an effective outcome.
- ✓ We will find correlations, general trends, and outliers.

1.3. Data Wrangling

- ✓ Data wrangling is the process of cleaning and converting raw data into a useable format.
- ✓ It is the process of,
 - Cleaning the data
 - Missing Values
 - Duplicate data
 - Invalid data
 - Selecting the variable to use,
- ✓ It is not necessary that data we have collected is always of our use as some of the data may not be useful.

1.4. Train the model

- ✓ During training, the model can learn different patterns and rules.
- ✓ We need to use train dataset to train the model.

1.5. Test the model

- ✓ Once model trained then testing is required to check the accuracy.
- ✓ We need to use test dataset to test the model.

1.6. Model deployment

- ✓ Once model trained and tested, if model is producing good results then we can deploy the model in the real time.

7. Data Science – Machine Learning – Train & Test Datasets

Contents

1. Types of Datasets in machine learning.....	2
2. Train dataset.....	3
3. Test dataset	3
4. How to decide size of these 3 sets?	3
5. Creating array	4
6. train_test_split(p) function.....	5
7. train_test_split(p, random_state = 0) function.....	18

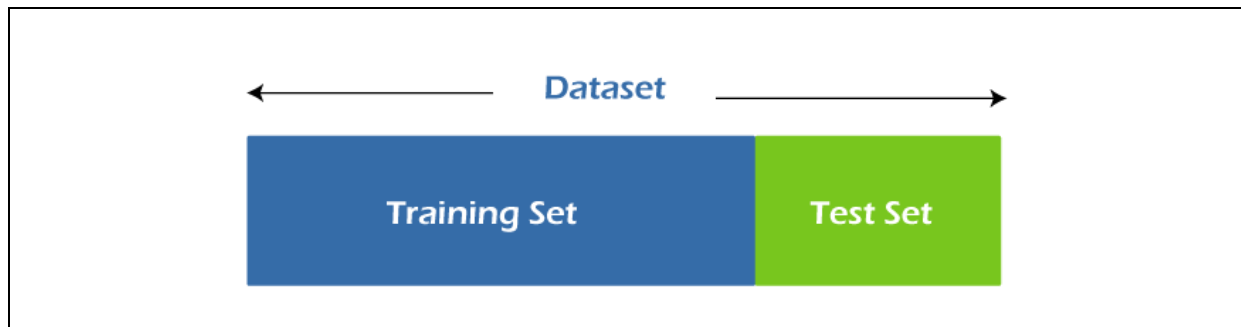
7. Data Science – Machine Learning – Train & Test Datasets

1. Types of Datasets in machine learning

- ✓ There are mainly 3 types of datasets used in Machine learning,
 - Train dataset
 - Test dataset
 - Validation dataset

Note

- ✓ Here validation dataset is an optional but training and test datasets are mandatory



2. Train dataset

- ✓ Train dataset is used to train the models.
- ✓ This is the part of dataset which is used to train the model.
- ✓ Typically, training set contains about 60-70% of total dataset.
- ✓ First step is, model should get train with training dataset, during training model learns the parameters and underlying concepts from dataset.

3. Test dataset

- ✓ Test dataset is used to test the models.
- ✓ Once model training is done then we need to test the model with test dataset.
- ✓ During testing the model we will understand about model performance either good or not.
- ✓ Size of Test set is about 15-30% of total dataset.

4. How to decide size of these 3 sets?

- ✓ There is no thumb rule about choosing the size of these sets, but according to the experts' 70-30 or 60-40 is a good size for train and test set respectively.

5. Creating array

- ✓ We can create an array and split that array

Program Name Creating an array
demo1.py

```
import numpy as np

dataset = np.arange(10)

print(dataset)
```

Output

```
[0 1 2 3 4 5 6 7 8 9]
```


6. train_test_split(p) function

- ✓ train_test_split(p) is a predefined function in sklearn.model_selection package
- ✓ We need to access this function from sklearn package.
- ✓ By using this function we can split the dataset into train dataset and test dataset.

Program Name Creating an array and splitting dataset
demo2.py

```
import numpy as np
from sklearn.model_selection import train_test_split

dataset = np.arange(10)

result = train_test_split(dataset)

print(dataset)
print(result)
```

Output

```
C:\Users\Nireekshan\Desktop\PROGRAMS>py demo1.py
[0 1 2 3 4 5 6 7 8 9]
[array([0, 1, 8, 9, 2, 5, 7]), array([3, 6, 4])]

C:\Users\Nireekshan\Desktop\PROGRAMS>py demo1.py
[0 1 2 3 4 5 6 7 8 9]
[array([3, 2, 6, 4, 1, 5, 9]), array([7, 8, 0])]

C:\Users\Nireekshan\Desktop\PROGRAMS>py demo1.py
[0 1 2 3 4 5 6 7 8 9]
[array([7, 8, 4, 3, 0, 6, 2]), array([5, 1, 9])]
```

Program Name Creating an array and splitting
demo3.py

```
import numpy as np
from sklearn.model_selection import train_test_split

dataset = np.arange(10)

X_train, X_test = train_test_split(dataset)

print(dataset)
print()
print(X_train)
print(X_test)
```

Output

```
[0 1 2 3 4 5 6 7 8 9]

[0 6 5 1 4 9 7]
[8 3 2]
```

Program Name Creating an array and splitting
demo4.py

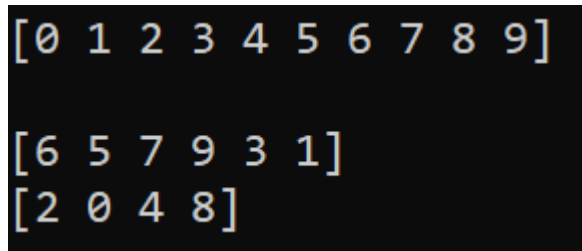
```
import numpy as np
from sklearn.model_selection import train_test_split

dataset = np.arange(10)

X_train, X_test = train_test_split(dataset, test_size = 4)

print(dataset)
print()
print(X_train)
print(X_test)
```

Output



```
[0 1 2 3 4 5 6 7 8 9]

[6 5 7 9 3 1]
[2 0 4 8]
```

Program Name Creating an array and splitting
demo5.py

```
import numpy as np
from sklearn.model_selection import train_test_split

dataset = np.arange(10)

X_train, X_test = train_test_split(dataset, train_size = 6)

print(dataset)
print()
print(X_train)
print(X_test)
```

Output

```
[0 1 2 3 4 5 6 7 8 9]

[1 9 4 3 2 5]
[7 8 6 0]
```

Program Name Creating an array and splitting
demo6.py

```
import numpy as np
from sklearn.model_selection import train_test_split

dataset = np.arange(10)

X_train, X_test = train_test_split(dataset, test_size = 0.4)

print(dataset)
print()
print(X_train)
print(X_test)
```

Output

```
[0 1 2 3 4 5 6 7 8 9]

[9 5 2 0 8 7]
[1 4 3 6]
```

Program Name Creating an array and splitting
demo7.py

```
import numpy as np
from sklearn.model_selection import train_test_split

dataset = np.arange(10)

X_train, X_test = train_test_split(dataset, train_size = 0.6)

print(dataset)
print()
print(X_train)
print(X_test)
```

Output

```
[0 1 2 3 4 5 6 7 8 9]

[9 6 4 8 3 5]
[2 1 0 7]
```

Program Name Creating an array and splitting
demo8.py

```
import numpy as np
from sklearn.model_selection import train_test_split

dataset = np.arange(10)

X_train, X_test = train_test_split(dataset, train_size = 4, test_size
= 4)

print(dataset)
print()
print(X_train)
print(X_test)
```

Output

```
[0 1 2 3 4 5 6 7 8 9]
[0 9 3 6]
[4 7 2 5]
```

Program Name Creating an array and splitting
demo9.py

```
import numpy as np
from sklearn.model_selection import train_test_split

dataset = np.arange(10)

X_train, X_test = train_test_split(dataset, train_size = 4, test_size
= 10)

print(dataset)
print()
print(X_train)
print(X_test)
```

Output

```
ValueError: test_size=10 should be either positive and smaller than the number of samples 10 or a float
in the (0, 1) range
```


Program Name Creating an array and splitting
demo10.py

```
import numpy as np

dataset = np.arange(20)

print(dataset)
```

Output

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
```

Program Name Creating an array and splitting
demo11.py

```
import numpy as np

dataset = np.arange(20).reshape(2, 10)

print(dataset)
```

Output

```
[[ 0  1  2  3  4  5  6  7  8  9]
 [10 11 12 13 14 15 16 17 18 19]]
```

Program Name Creating an array and splitting
demo12.py

```
import numpy as np

dataset = np.arange(20).reshape(2, 10).T

print(dataset)
```

Output

```
[[ 0 10]
 [ 1 11]
 [ 2 12]
 [ 3 13]
 [ 4 14]
 [ 5 15]
 [ 6 16]
 [ 7 17]
 [ 8 18]
 [ 9 19]]
```

Program Name Creating an array and splitting
demo13.py

```
import numpy as np

X = np.arange(20).reshape(2, 10).T

y = np.arange(10)

print(X)
print()
print(y)
```

Output

```
[[ 0 10]
 [ 1 11]
 [ 2 12]
 [ 3 13]
 [ 4 14]
 [ 5 15]
 [ 6 16]
 [ 7 17]
 [ 8 18]
 [ 9 19]]

[0 1 2 3 4 5 6 7 8 9]
```

Program Name Creating an array and splitting
demo14.py

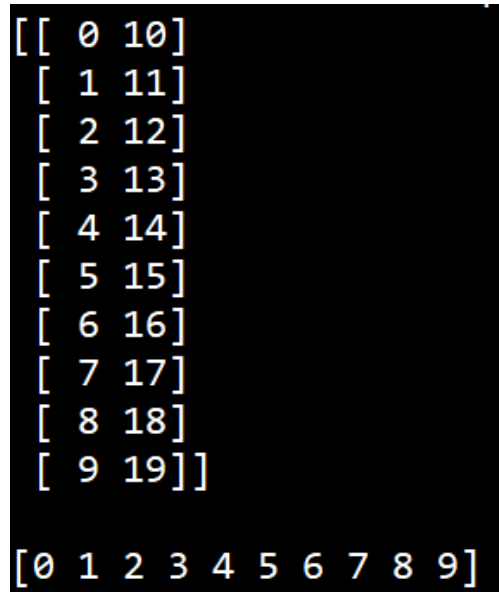
```
import numpy as np

X = np.arange(20).reshape(2, 10).T

y = np.arange(10)

print(X)
print()
print(y)
```

Output



```
[[ 0 10]
 [ 1 11]
 [ 2 12]
 [ 3 13]
 [ 4 14]
 [ 5 15]
 [ 6 16]
 [ 7 17]
 [ 8 18]
 [ 9 19]]

[0 1 2 3 4 5 6 7 8 9]
```

Program Name Creating an array and splitting
demo15.py

```
import numpy as np
from sklearn.model_selection import train_test_split

X = np.arange(20).reshape(2, 10).T

y = np.arange(10)

X_train, X_test, y_train, y_test = train_test_split(X, y)

print(X_train)
print()
print(X_test)
print()
print(y_train)
print()
print(y_test)
```

Output

```
[[ 9 19]
 [ 6 16]
 [ 2 12]
 [ 4 14]
 [ 3 13]
 [ 8 18]
 [ 7 17]]

[[ 5 15]
 [ 1 11]
 [ 0 10]]

[9 6 2 4 3 8 7]

[5 1 0]
```

7. `train_test_split(p, random_state = 0)` function

- ✓ `train_test_split(p, random_state = 0)` is a predefined function in `sklearn.model_selection` package
- ✓ We need to access this function from `sklearn` package.
- ✓ By using this function we can split the dataset into train dataset and test dataset.
- ✓ We will get the same train and test datasets across different executions.

Program Name Creating an array and splitting
demo16.py

```
import numpy as np
from sklearn.model_selection import train_test_split

X = np.arange(20).reshape(2, 10).T

y = np.arange(10)

X_train, X_test, y_train, y_test = train_test_split(X, y,
random_state = 0)

print(X_train)
print()
print(X_test)
print()
print(y_train)
print()
print(y_test)
```

Output

```
[[ 9 19]
 [ 1 11]
 [ 6 16]
 [ 7 17]
 [ 3 13]
 [ 0 10]
 [ 5 15]]

[[ 2 12]
 [ 8 18]
 [ 4 14]]

[9 1 6 7 3 0 5]

[2 8 4]
```

8. Data Science – Machine Learning – R value

Contents

1. Regression	2
2. A line	2
3. The goal	3
4. Can we use regression in everywhere?	4
5. r value	4
6. r value range.....	4
7. Calculate r value	5

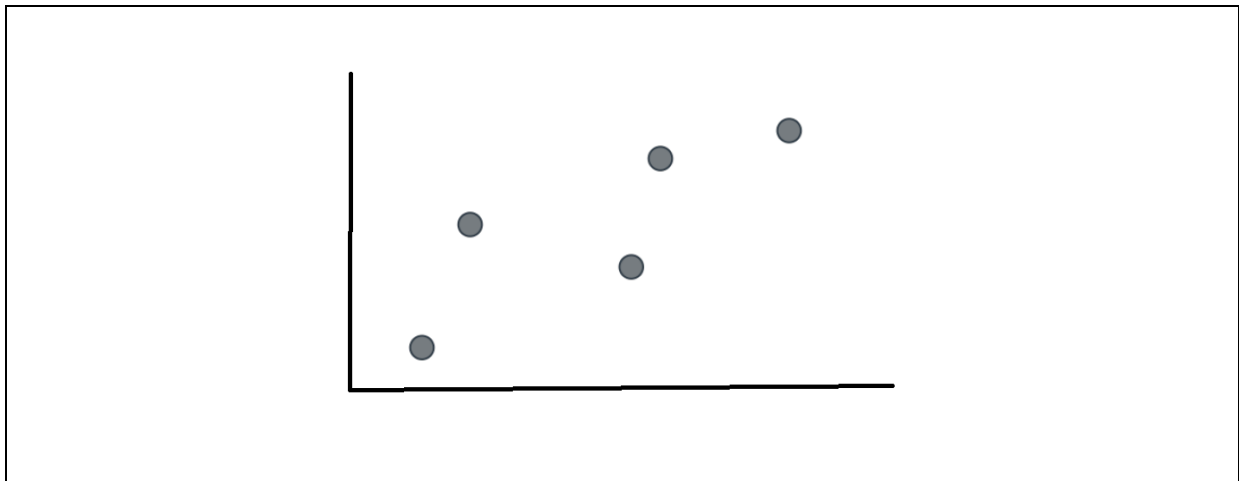
8. Data Science – Machine Learning – R value

1. Regression

- ✓ By using regression we can find the relationship between variables.
- ✓ Once we understand the relationship in between the variables then we can predict the future outcomes

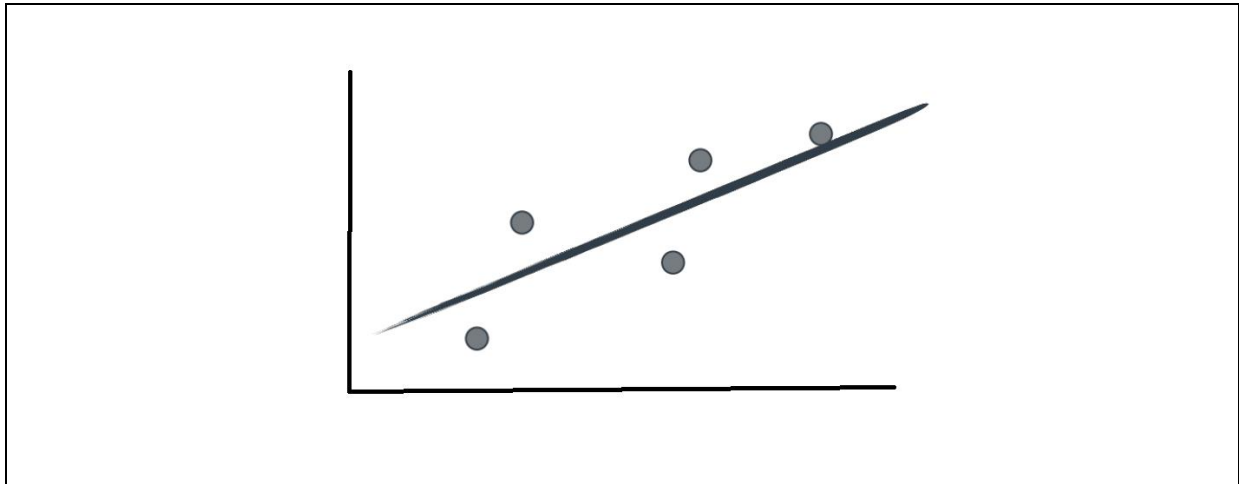
2. A line

- ✓ If two variables having relationship then if we draw this relationship in a two dimensional then we get a straight line.
- ✓ The picture of linear regression is simple.
- ✓ Let us say we have some points, a line will travel in between these points



3. The goal

- ✓ The goal of linear regression is to draw the best fitted line.
- ✓ Best fitted line means that the line which passes as close as possible to these points.



4. Can we use regression in everywhere?

- ✓ If there is a relationship in between the variables then only we can use linear regression algorithm.
- ✓ If there is no relationship in between the variables then we cannot use linear regression algorithm.

5. r value

- ✓ r value explains about how variables are related each other.
- ✓ This is very important step to recognise relationship in between values of x-axis and y-axis.

6. r value range

- ✓ The r values range, from -1 to 1.
- ✓ While calculating if we get r value as near to **-1** or **1** then those variables are **strongly related** each other.
- ✓ While calculating if we get r value as **0** then it confirms that there is **no relationship** in between the variables.

7. Calculate r value

- ✓ By using python scipy module we can calculate r value easily

Program Name Plotting x and y values
demo1.py

```
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt

d = {
    "area": [1, 2, 3, 4],
    "rice_yield": [10, 20, 30, 40]
}

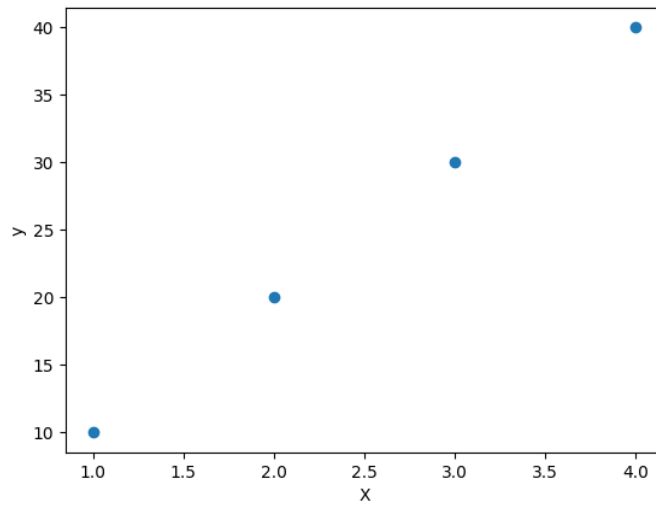
df = pd.DataFrame(d)

X = df.area.values
y = df.rice_yield.values

plt.xlabel("X")
plt.ylabel("y")

plt.scatter(X, y)
plt.show()
```

Output



Program Name Calculating r value
demo2.py

```
import pandas as pd
from scipy import stats

d = {
    "area": [1, 2, 3, 4],
    "rice_yield": [10, 20, 30, 40]
}

df = pd.DataFrame(d)

X = df.area.values
y = df.rice_yield.values

slope, intercept, r_value, p_value, std_err = stats.linregress(X, y)

print(r_value)
```

Output

1.0

Note:

- ✓ The result 1.0
- ✓ So we can confirm that there is strong a relationship.
- ✓ So we can apply linear regression algorithm on top of these variables for future prediction

Program Name Calculating r value
demo3.py

```
import pandas as pd
from scipy import stats

d = {
    "a": [600, 3000, 2, 3600, 4],
    "b": [550000, 565000, 610000, 680000, 725000]
}

df = pd.DataFrame(d)

X = df.a.values
y = df.b.values

slope, intercept, r_value, p_value, std_err = stats.linregress(X, y)

print(r_value)
```

Output

-0.06533879637370224

Note:

- ✓ The result -0.06533879637370224 this value is very near to 0
- ✓ So we can confirm that there is no relationship.
- ✓ So we cannot apply linear regression algorithm on top of these variables, even if we apply then we will get wrong prediction results

9. Data Science – Machine Learning – Simple Linear Regression

Contents

1. Regression	2
2. A line	2
3. The goal	3
4. Linear Regression	4
5. Types of linear regression	4
6. Simple linear regression	4
7. Multiple linear regression	4
8. Simple linear regression example	5
8.1. Problem statement	5
8.2. The solution	5
9. Machine learning Terminology	6
9.1. Features	6
9.2. Label or target	6
9.3. Models	6
9.4. Prediction	6
9.5. Formula	6
10. Intercept and coefficient	17
11. $Y = m * X + b$ (m is coefficient and b is intercept)	18
12. Best fitted line	19
13. Predicting a group of home prices	21

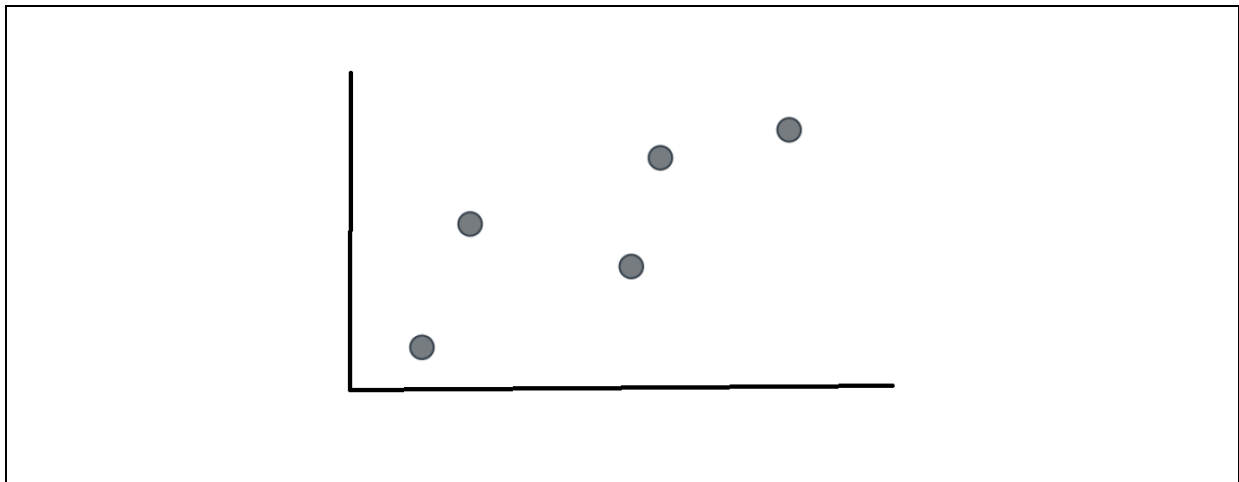
9. Data Science – Machine Learning – Simple Linear Regression

1. Regression

- ✓ Regression analysis is used to explain the relationship between a two variables.
- ✓ Also called as it's a relationship in between dependent variable and one or more independent variables.

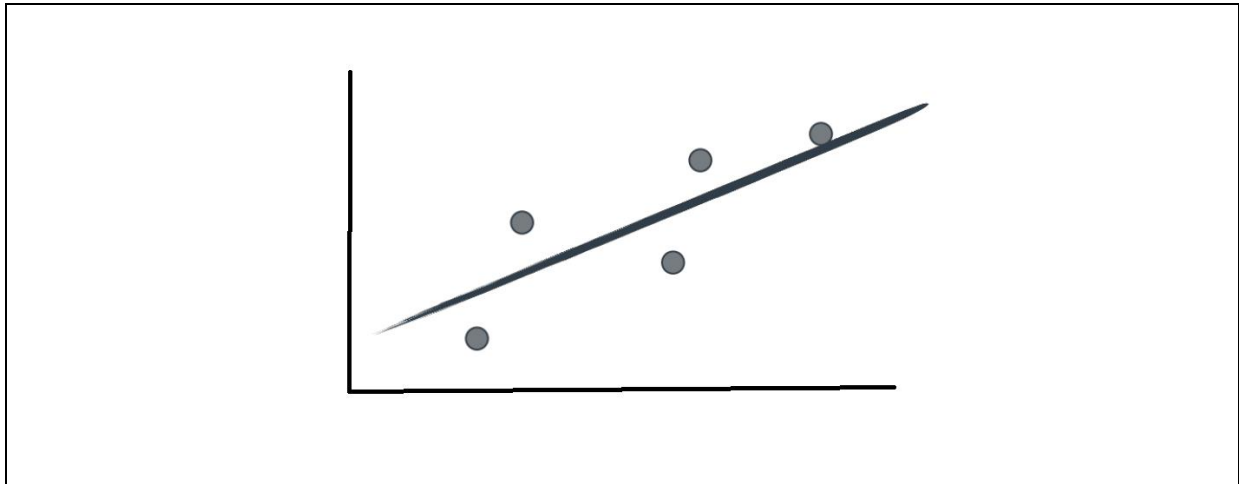
2. A line

- ✓ If two variables having relationship then if we draw this relationship in a two dimensional then we get a straight line.
- ✓ The picture of linear regression is simple.
- ✓ Let us say we have some points, a line will travel in between these points



3. The goal

- ✓ The goal of linear regression is to draw the best fitted line.
- ✓ Best fitted line means that the line which passes as close as possible to these points.



4. Linear Regression

- ✓ This is a technique and it explains the relationship between the dependent variable and independent variables

5. Types of linear regression

- ✓ There are two types of linear regression
 - Simple linear regression
 - Multiple linear regression

6. Simple linear regression

- ✓ When you have only 1 independent variable and 1 dependent variable, it is called simple linear regression.

7. Multiple linear regression

- ✓ When you have 2 or more independent variable and 1 dependent variable, it is called multiple linear regression.

8. Simple linear regression example

- ✓ When you have only 1 independent variable and 1 dependent variable, it is called simple linear regression.

8.1. Problem statement

- ✓ Assuming that we are planning to buy a new house and need to predict the price of a house

8.2. The solution

- ✓ While buying house first we need to check the area of the house

area	price
2600	550000
3000	565000
3200	610000
3600	680000
4000	725000

9. Machine learning Terminology

9.1. Features

- ✓ From the given problem the feature is **area** and **price**

9.2. Label or target

- ✓ Price of the house

9.3. Models

- ✓ A machine learning model is simply a rule, or a formula, which predicts a label from the features.
- ✓ In this case, the model is the equation we found for the price.

9.4. Prediction

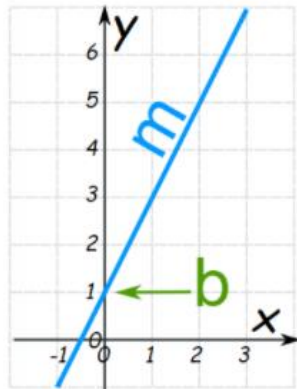
- ✓ The prediction is simply the output of the model.
- ✓ If the model gives the result as “Hey Guru I think the house with 36000 area is going to cost \$300”, then the prediction is 300.

9.5. Formula

- ✓ Home price = $m * (\text{area}) + b$

Reminder

- ✓ Once please walk through our maths regression chapter (**Chapter 7. Statistics - PART - 7 - Regression**) which we have already discussed, thanks



$$\text{price} = m * \text{area} + b$$

$$y = mX + b$$

Slope (or Gradient) Y Intercept

Program Name Loading house prices dataset
demo1.py

```
import pandas as pd

df = pd.read_csv("homeprices.csv")

print(df.head())
```

Output

	area	price
0	2600	550000
1	3000	565000
2	3200	610000
3	3600	680000
4	4000	725000

Program Name Creating scatter plot using matplotlib
demo2.py

```
import pandas as pd
import matplotlib.pyplot as plt

df=pd.read_csv("homeprices.csv")

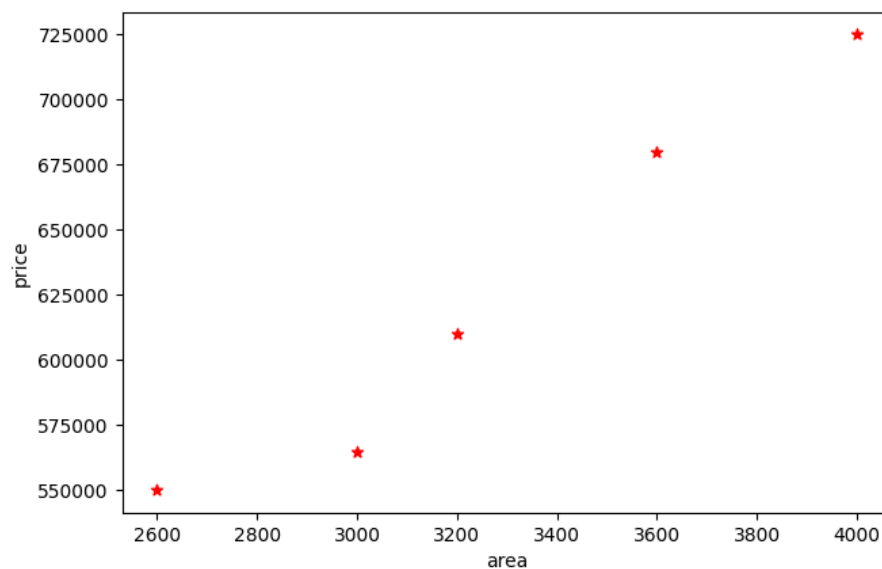
# plotting the dataset

plt.xlabel('area')
plt.ylabel('price')

plt.scatter(df.area, df.price, color = 'red', marker = '*')

plt.show()
```

Output



Program Name Loading the data set
demo3a.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis = 'columns')

print(df)
print()
print(new_df)
```

Output

```
   area  price
0  2600  550000
1  3000  565000
2  3200  610000
3  3600  680000
4  4000  725000

   area
0  2600
1  3000
2  3200
3  3600
4  4000
```

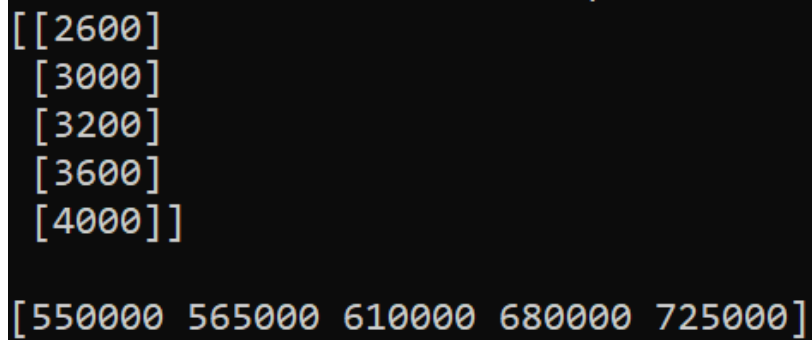
Program Name Loading the data set
demo3b.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis = 'columns')

print(new_df.values)
print()
print(df.price.values)
```

Output



```
[[2600]
 [3000]
 [3200]
 [3600]
 [4000]]

[550000 565000 610000 680000 725000]
```

Program Name Creating LinearRegression object
demo3.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis = 'columns')

# Training the Algorithm
reg = LinearRegression()
reg.fit(new_df.values, df.price.values)

print("Training the Algorithm")
```

Output

Training the Algorithm

Program Name Predict price of a home with area = 3300 sqr ft
demo4.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df=pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis='columns')

reg = LinearRegression()
reg.fit(new_df.values, df.price.values)

# Making Predictions
print(reg.predict([[3300]]))
```

Output

```
[628715.75342466]
```

Program Name Predict price of a home with area = 5000 sqr ft
demo5.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df=pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis='columns')

reg = LinearRegression()
reg.fit(new_df.values, df.price.values)

# Making Predictions
print(reg.predict([[5000]]))
```

Output

```
[859554.79452055]
```

Program Name Capture the coefficient from regression
demo6.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df=pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis='columns')

reg = LinearRegression()
reg.fit(new_df.values, df.price.values)

print(reg.coef_)
```

Output

```
[135.78767123]
```

Program Name Capture the intercept from regression
demo7.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df=pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis='columns')

reg = LinearRegression()
reg.fit(new_df.values, df.price.values)

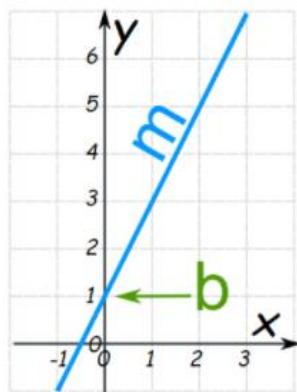
print(reg.intercept_)
```

Output

180616.43835616432

10. Intercept and coefficient

- ✓ Intercept = 180616.43835616432
- ✓ Coefficient = 135.78767123



$$\text{price} = m * \text{area} + b$$

$$y = mX + b$$

Slope (or Gradient) Y Intercept

11. $Y = m * X + b$ (m is coefficient and b is intercept)

- ✓ Let's calculate the above formula.
- ✓ In the given formula m is coefficient and b is intercept
- ✓ $Y = m * X + b$
- ✓ $Y = 135.78767123 * 3300 + 180616.43835616432$
- ✓ $Y = 628715.75342466$
- ✓ Awesome....!!!!

12. Best fitted line

- ✓ Let's calculate the above formula.
- ✓ We can draw a line

Program Name Drawing a best fitted line
demo8.py

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

df=pd.read_csv("homeprices.csv")

new_df = df.drop('price', axis='columns')

reg = LinearRegression()
reg.fit(new_df.values, df.price.values)

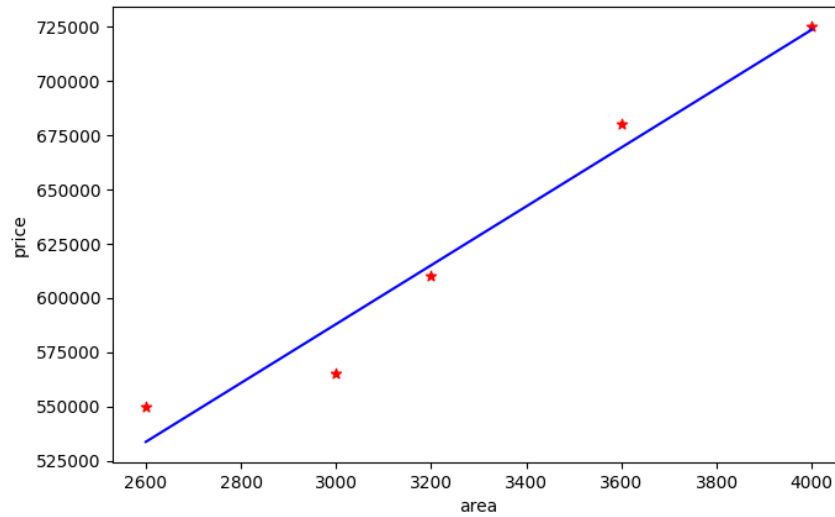
plt.xlabel('area')
plt.ylabel('price')

plt.scatter(df.area.values, df.price.values, color = 'red', marker =
'*)

plt.plot(df.area.values, reg.predict(df[['area']].values), color =
'blue')

plt.show()
```

Output



13. Predicting a group of home prices

- ✓ By using above model we can predict the group of home prices as well

Program Loading a group of house areas
Name demo9.py

```
import pandas as pd
from sklearn import linear_model
from sklearn.linear_model import LinearRegression

df = pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis='columns')

reg = LinearRegression()
reg.fit(new_df.values, df.price.values)

area_df = pd.read_csv("areas.csv")
print(area_df)
```

Output

```
   area
0  1000
1  1500
2  2300
3  3540
4  4120
5  4560
6  5490
7  3460
8  4750
9  2300
10 9000
11 8600
12 7100
```

Program Name Predicting a group of home prices
demo10.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis='columns')

reg = LinearRegression()
reg.fit(new_df.values, df.price.values)

area_df = pd.read_csv("areas.csv")

prices = reg.predict(area_df.values)
print(prices)
```

Output

```
[ 316404.10958904  384297.94520548  492928.08219178  661304.79452055
 740061.64383562  799808.21917808  926090.75342466  650441.78082192
 825607.87671233  492928.08219178 1402705.47945205 1348390.4109589
1144708.90410959]
```

Program Name Create a csv file with predictions
demo11.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis='columns')

reg = LinearRegression()
reg.fit(new_df.values, df.price.values)

area_df = pd.read_csv("areas.csv")
p = reg.predict(area_df.values)

area_df['prices'] = p
area_df.to_csv('output.csv')
print("Please check in current directory for output.csv")
```

Output

Please check in current directory for output.csv

9.1. Data Science – Machine Learning – Linear Regression Example

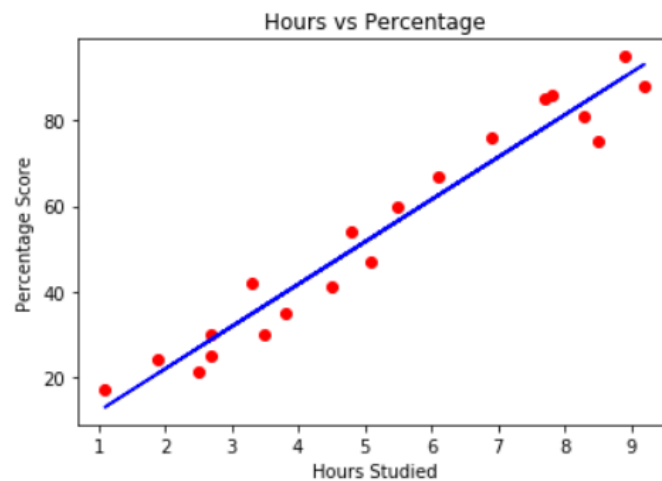
Contents

1. Scenario	2
2. Maths behind	3
3. Loading dataset	4
4. Plotting the data	5
5. Intercept & coefficient	10
6. Important info	12
7. Making Predictions.....	12
8. Evaluating the Algorithm	15
9. Evaluation metrics.....	15

9.1. Data Science – Machine Learning – Linear Regression Example

1. Scenario

- ✓ Let's find the relationship in between **marks** and **number of study hours**
- ✓ We want to find out the **marks** for given **number of study hours** to a student
- ✓ If we plot the independent variable (hours) on the x-axis and dependent variable (percentage) on the y-axis then linear regression gives us a straight line that best fits the data points.



2. Maths behind

- ✓ We know that the equation of a straight line is basically
 - $y = mx + b$
- ✓ Where **b** is the **intercept** and **m** is the **slope** of the line.
- ✓ So basically, the linear regression algorithm gives us the most optimal value for the **intercept** and the **slope**.
- ✓ The y and x variables remain the same
- ✓ There can be multiple straight lines depending upon the values of intercept and slope.
- ✓ Basically what the linear regression algorithm does is it fits multiple lines on the data points and returns the line that results in the least error.

3. Loading dataset

- ✓ Once we have dataset then we need to load by using pandas
- ✓ If we load dataset then it returns the DataFrame

Program Name Loading student_scores dataset
demo1.py

```
import pandas as pd

df = pd.read_csv('student_scores.csv')

print(df.head())
```

Output

	Hours	Scores
0	2.5	21
1	5.1	47
2	3.2	27
3	8.5	75
4	3.5	30

4. Plotting the data

- ✓ Let's plot our data points to see the relationship between the data.

Program Name Plotting the dataset
demo2.py

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('student_scores.csv')

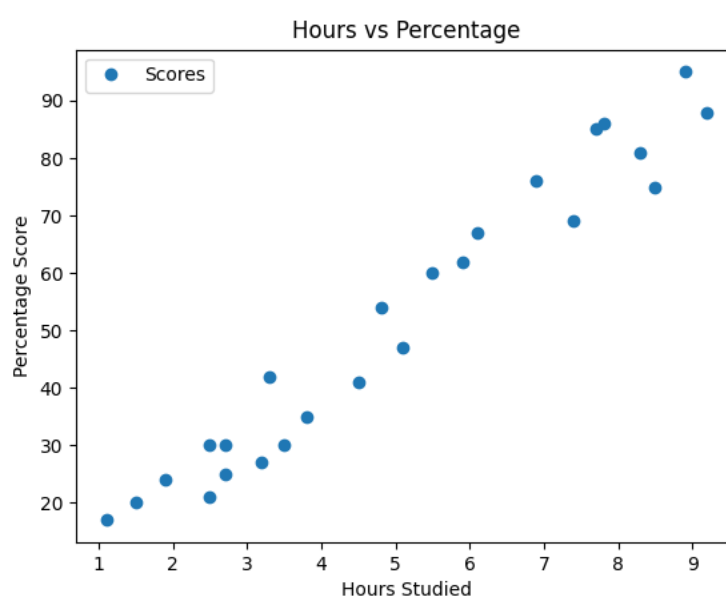
df.plot(x='Hours', y='Scores', style='o')

plt.title('Hours vs Percentage')

plt.xlabel('Hours Studied')
plt.ylabel('Percentage Score')

plt.show()
```

Output



Program Name Preparing the data
demo3.py

```
import pandas as pd

df = pd.read_csv('student_scores.csv')

X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values

print(X)
print(y)
```

Output

```
[[2.5]
 [5.1]
 [3.2]
 [8.5]
 [3.5]
 [1.5]
 [9.2]
 [5.5]
 [8.3]
 [2.7]
 [7.7]
 [5.9]
 [4.5]
 [3.3]
 [1.1]
 [8.9]
 [2.5]
 [1.9]
 [6.1]
 [7.4]
 [2.7]
 [4.8]
 [3.8]
 [6.9]
 [7.8]]
[21 47 27 75 30 20 88 60 81 25 85 62 41 42 17 95 30 24 67 69 30 54 35 76
 86]
```

Program Splitting the data
Name demo4.py

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

df = pd.read_csv('student_scores.csv')

X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
0.2, random_state = 0)

print("X_train")
print(X_train)

print()
print("X_test")
print(X_test)

print()
print("Y_train")
print(y_train)

print()
print("y_test")
print(y_test)
```

Output

```
X_train
[[3.8]
 [1.9]
 [7.8]
 [6.9]
 [1.1]
 [5.1]
 [7.7]
 [3.3]
 [8.3]
 [9.2]
 [6.1]
 [3.5]
 [2.7]
 [5.5]
 [2.7]
 [8.5]
 [2.5]
 [4.8]
 [8.9]
 [4.5]]

X_test
[[1.5]
 [3.2]
 [7.4]
 [2.5]
 [5.9]]

Y_train
[35 24 86 76 17 47 85 42 81 88 67 30 25 60 30 75 21 54 95 41]

y_test
[20 27 69 30 62]
```

Program Name Training the model
demo5.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv('student_scores.csv')

X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
0.2, random_state=0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

print("Model got trained with data")
```

Output

Model got trained with data

5. Intercept & coefficient

- ✓ In the theory section we said that linear regression model basically finds the best value for the intercept and slope, which results in a line that best fits the data.
- ✓ We can get the values of the intercept and slop from linear regression

Program Name Getting intercept from created model
demo6.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv('student_scores.csv')

X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

print(regressor.intercept_)
```

Output

2.018160041434669

Program Name Getting coefficient from created model
demo7.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv('student_scores.csv')

X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
0.2, random_state = 0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

print(regressor.coef_)
```

Output

```
[9.91065648]
```

6. Important info

- ✓ This means that for every one unit of change in hours studied, the change in the score is about 9.91%.
- ✓ In simpler words, if a student studies **one hour more** than they previously studied for an exam, they can expect to achieve an increase of **9.91%** in the **score** achieved by the student previously.

7. Making Predictions

- ✓ Now successfully we have trained our algorithm.
- ✓ So, it's time to make some predictions.
- ✓ We need to use our test data and see how accurately our algorithm predicts the percentage score.

Program Name Making predictions
demo8.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv('student_scores.csv')

X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)
print(y_pred)
```

Output

```
[16.88414476 33.73226078 75.357018 26.79480124
60.49103328]
```

Great

- ✓ The y_pred is a numpy array that contains all the predicted values for the input values in the X_test series.

Comparison

- ✓ To compare the actual output values for X_test with the predicted values.

Program Name Comparing the predicted result
demo9.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv('student_scores.csv')

X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

d = {'Actual': y_test, 'Predicted': y_pred}

compare_df = pd.DataFrame(d)
print(compare_df)
```

Output

	Actual	Predicted
0	20	16.884145
1	27	33.732261
2	69	75.357018
3	30	26.794801
4	62	60.491033

- ✓ Though our model is not very precise, the predicted percentages are close to the actual ones.

8. Evaluating the Algorithm

- ✓ We need to evaluate the performance of algorithm.
- ✓ This step is particularly important to compare how well different algorithms perform on a particular dataset.
- ✓ For regression algorithms there are three evaluation metrics are commonly used

9. Evaluation metrics

- ✓ Mean Absolute Error (**MAE**)
- ✓ Mean Squared Error (**MSE**)
- ✓ Root Mean Squared Error (**RMSE**)

Mean Absolute Error (MAE)

- ✓ Mean Absolute Error (**MAE**) is the mean of the absolute value of the errors.

Mean Squared Error (MSE)

- ✓ Mean Squared Error (**MSE**) is the mean of the squared errors.

Root Mean Squared Error (RMSE)

- ✓ Root Mean Squared Error (**RMSE**) is the square root of the mean of the squared errors

$$MAE = \frac{1}{n} \sum \left| y - \hat{y} \right|$$

Diagram annotations for MAE:

- Divide by the total number of data points**: Points to the $\frac{1}{n}$ term.
- Actual output value**: Points to the y term inside the absolute value.
- Predicted output value**: Points to the $\hat{y}$ term inside the absolute value.
- Sum of**: Points to the summation symbol Σ .
- The absolute value of the residual**: Points to the absolute value bars $| \cdot |$.

$$MSE = \frac{1}{n} \sum \left(y - \hat{y} \right)^2$$

Diagram annotation for MSE:

- The square of the difference between actual and predicted**: Points to the term $(y - \hat{y})^2$.

We no need to calculate manually

- ✓ We don't have to perform these calculations manually.
- ✓ The scikit-Learn library comes with pre-built functions that can be used to find out these values for us.

Program Name Loading student_scores dataset
demo10.py

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics

df = pd.read_csv('student_scores.csv')

X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

mae = metrics.mean_absolute_error(y_test, y_pred)
mse = metrics.mean_squared_error(y_test, y_pred)
rmse = np.sqrt(metrics.mean_squared_error(y_test, y_pred))

print('Mean Absolute Error:', mae)
print('Mean Squared Error:', mse)
print('Root Mean Squared Error:', rmse)
```

Output

Mean Absolute Error: 4.18385989900298

Mean Squared Error: 21.598769307217413

Root Mean Squared Error: 4.647447612100368

9.2. Data Science – Machine Learning – Linear Regression Example

Program Loading salary dataset
Name demo1.py

```
import pandas as pd

dataset = pd.read_csv('Salary_Data.csv')

print(dataset.head())
```

Output

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

Program Name Preparing the data
demo2.py

```
import pandas as pd

dataset = pd.read_csv('Salary_Data.csv')

X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 1].values

print(X)
print(y)
```

Output

```
[[ 1.1]
 [ 1.3]
 [ 1.5]
 [ 2. ]
 [ 2.2]
 [ 2.9]
 [ 3. ]
 [ 3.2]
 [ 3.2]
 [ 3.7]
 [ 3.9]
 [ 4. ]
 [ 4. ]
 [ 4.1]
 [ 4.5]
 [ 4.9]
 [ 5.1]
 [ 5.3]
 [ 5.9]
 [ 6. ]
 [ 6.8]
 [ 7.1]
 [ 7.9]
 [ 8.2]
 [ 8.7]
 [ 9. ]
 [ 9.5]
 [ 9.6]
 [10.3]
 [10.5]]
[ 39343.  46205.  37731.  43525.  39891.  56642.  60150.  54445.  64445.
  57189.  63218.  55794.  56957.  57081.  61111.  67938.  66029.  83088.
  81363.  93940.  91738.  98273. 101302. 113812. 109431. 105582. 116969.
112635. 122391. 121872.]
```

Program Name Splitting the dataset
demo3.py

```
import pandas as pd
from sklearn.model_selection import train_test_split

dataset = pd.read_csv('Salary_Data.csv')

X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
1/3, random_state = 0)

print("X_train")
print(X_train)
print()

print("X_test")
print(X_test)
print()

print("y_train")
print(y_train)
print()

print("y_test")
print(y_test)
```

Output

```
X_train
[[ 2.9]
 [ 5.1]
 [ 3.2]
 [ 4.5]
 [ 8.2]
 [ 6.8]
 [ 1.3]
 [10.5]
 [ 3. ]
 [ 2.2]
 [ 5.9]
 [ 6. ]
 [ 3.7]
 [ 3.2]
 [ 9. ]
 [ 2. ]
 [ 1.1]
 [ 7.1]
 [ 4.9]
 [ 4. ]]
```

```
X_test
[[ 1.5]
 [10.3]
 [ 4.1]
 [ 3.9]
 [ 9.5]
 [ 8.7]
 [ 9.6]
 [ 4. ]
 [ 5.3]
 [ 7.9]]
```

```
y_train
[ 56642.  66029.  64445.  61111. 113812.  91738.  46205. 121872.  60150.
 39891.  81363.  93940.  57189.  54445. 105582.  43525.  39343.  98273.
 67938.  56957.]
```

```
y_test
[ 37731. 122391.  57081.  63218. 116969. 109431. 112635.  55794.  83088.
101302.]
```

Program Name Training the model
demo4.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

dataset = pd.read_csv('Salary_Data.csv')

X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
1/3, random_state = 0)

print("Training the model")

regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

Output

Training the model

Program Name Predicting the salaries
demo5.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

dataset = pd.read_csv('Salary_Data.csv')

X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
1/3, random_state = 0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

print("Predicting the salaries")

y_pred = regressor.predict(X_test)

print()
print(y_pred)
```

Output

```
Predicting the salaries
[ 40835.10590871 123079.39940819  65134.55626083  63265.36777221
 115602.64545369 108125.8914992  116537.23969801  64199.96201652
  76349.68719258 100649.1375447 ]
```

Program Name Plotting training dataset
demo6.py

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

dataset = pd.read_csv('Salary_Data.csv')

X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
1/3, random_state = 0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

print("Visualizing Training dataset results ")

plt.scatter(X_train, y_train, color = 'red')
plt.plot(X_train, regressor.predict(X_train), color = 'blue')

plt.title('Salary vs Experience {Training set}')
plt.xlabel('Years of experience')
plt.ylabel('Salary')

plt.show()
```

Output

Visualizing Training dataset results



Program Name Plotting test dataset
demo7.py

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

dataset = pd.read_csv('Salary_Data.csv')

X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 1].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size =
1/3, random_state = 0)

regressor = LinearRegression()
regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

print("Visualizing Training dataset results ")

plt.scatter(X_test, y_test, color = 'red')
plt.plot(X_train, regressor.predict(X_train), color = 'blue')

plt.title('Salary vs Experience {Test set}')
plt.xlabel('Years of experience')
plt.ylabel('Salary')

plt.show()
```

Output

Visualizing Training dataset results



10. Data Science – Machine Learning – Polynomial Features

Contents

1. Polynomial Features for machine learning	2
2. Need of Polynomial Features.....	2
3. Equations	3
4. Lets create Polynomial features	19

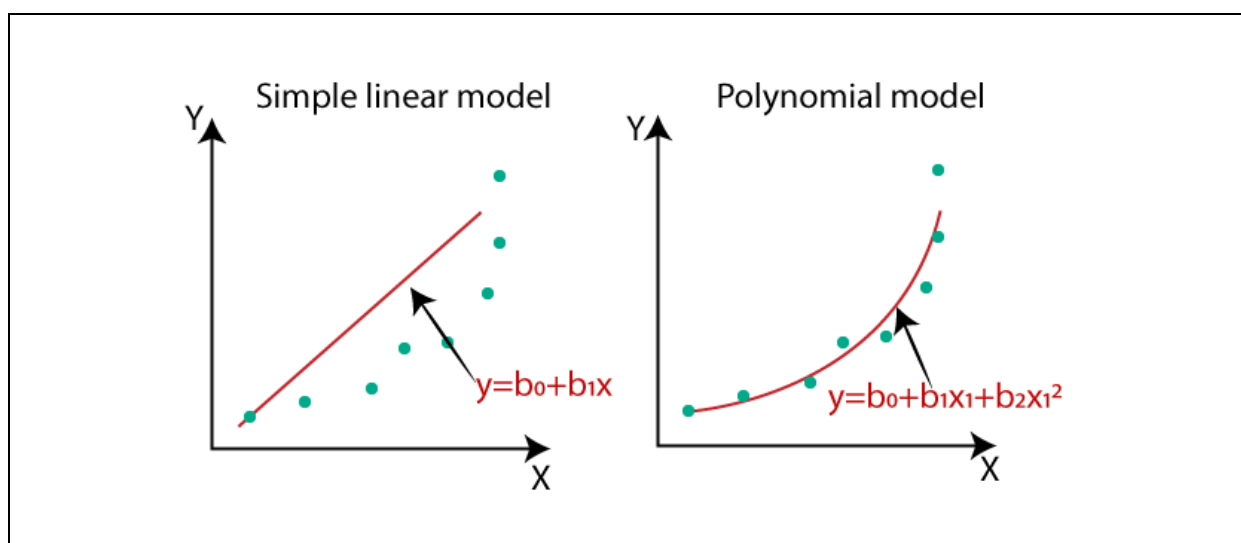
10. Data Science – Machine Learning – Polynomial Features

1. Polynomial Features for machine learning

- ✓ As such, polynomial features are a type of feature engineering means creating new input features based on the existing features.
- ✓ For example, if a dataset had one input feature X , then a polynomial feature would be the addition of a new feature (column) where values were calculated by squaring the values in X , e.g. X^2 .
- ✓ This process can be repeated for each input variable in the dataset, creating a transformed version of each.
- ✓ The "degree" of the polynomial is used to control the number of features added.

2. Need of Polynomial Features

- ✓ If we apply a linear model on a linear dataset, then it provides us a good result as we have seen in Simple Linear Regression.
- ✓ If we apply the same model without any modification on a non-linear dataset, then it will produce wrong results
- ✓ Due to this,
 - The error rate will be high etc



3. Equations

Simple Linear Regression equation

✓ $y = b_0 + b_1x$

Multiple Linear Regression equation

✓ $y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$

Polynomial Regression equation: $y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$

Program Name Creating an array
demo1.py

```
from numpy import asarray
```

```
data = asarray([[2], [3], [4]])  
print(data)
```

Output

```
[[2]  
 [3]  
 [4]]
```

Program Name Creating feature from existing feature
demo2.py

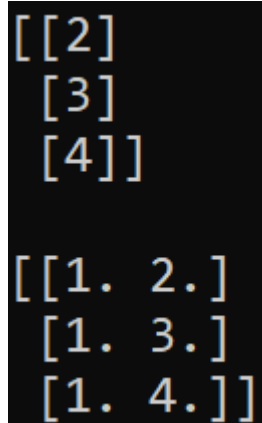
```
from numpy import asarray
from sklearn.preprocessing import PolynomialFeatures

data1 = asarray([[2],[3],[4]])

trans = PolynomialFeatures(degree = 1)
data2 = trans.fit_transform(data1)

print(data1)
print()
print(data2)
```

Output



```
[[2]
 [3]
 [4]]

[[1.  2.]
 [1.  3.]
 [1.  4.]]
```

Program Name Creating feature from existing feature
demo3.py

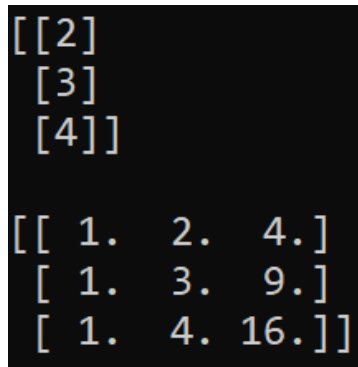
```
from numpy import asarray
from sklearn.preprocessing import PolynomialFeatures

data1 = asarray([[2],[3],[4]])

trans = PolynomialFeatures(degree = 2)
data2 = trans.fit_transform(data1)

print(data1)
print()
print(data2)
```

Output



```
[[2]
 [3]
 [4]]

[[ 1.  2.  4.]
 [ 1.  3.  9.]
 [ 1.  4. 16.] ]
```


Program Name Creating feature from existing feature
demo4.py

```
from numpy import asarray
from sklearn.preprocessing import PolynomialFeatures

data1 = asarray([[2],[3],[4]])

trans = PolynomialFeatures(degree = 3)
data2 = trans.fit_transform(data1)

print(data1)
print()
print(data2)
```

Output

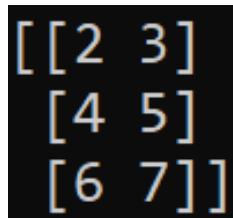
```
[[2]
 [3]
 [4]]

[[ 1.  2.  4.  8.]
 [ 1.  3.  9. 27.]
 [ 1.  4. 16. 64.]
```

Program Name Creating feature from existing feature
demo5.py

```
from numpy import asarray  
  
data1 = asarray([[2, 3],[4, 5],[6, 7]])  
  
print(data1)
```

Output



```
[[2 3]  
 [4 5]  
 [6 7]]
```

Program Name Creating feature from existing feature
demo6.py

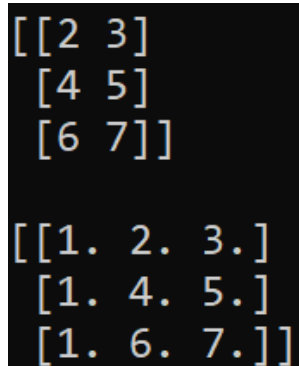
```
from numpy import asarray
from sklearn.preprocessing import PolynomialFeatures

data1 = asarray([[2, 3],[4, 5],[6, 7]])

trans = PolynomialFeatures(degree = 1)
data2 = trans.fit_transform(data1)

print(data1)
print()
print(data2)
```

Output



```
[[2 3]
 [4 5]
 [6 7]]

[[1.  2.  3.]
 [1.  4.  5.]
 [1.  6.  7.]]
```

Program Name Creating feature from existing feature
demo7.py

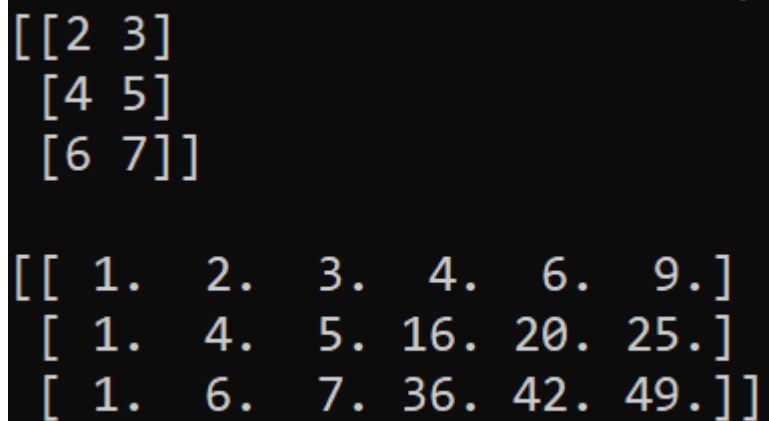
```
from numpy import asarray
from sklearn.preprocessing import PolynomialFeatures

data1 = asarray([[2, 3],[4, 5],[6, 7]])

trans = PolynomialFeatures(degree = 2)
data2 = trans.fit_transform(data1)

print(data1)
print()
print(data2)
```

Output



```
[[2 3]
 [4 5]
 [6 7]]

[[ 1.  2.  3.  4.  6.  9.]
 [ 1.  4.  5. 16. 20. 25.]
 [ 1.  6.  7. 36. 42. 49.]
```

Program Name Creating feature from existing feature
demo8.py

```
from numpy import asarray
from sklearn.preprocessing import PolynomialFeatures

data1 = asarray([[2, 3],[4, 5],[6, 7]])

trans = PolynomialFeatures(degree = 3)
data2 = trans.fit_transform(data1)

print(data1)
print()
print(data2)
```

Output

```
[[2 3]
 [4 5]
 [6 7]]

[[ 1.  2.  3.  4.  6.  9.  8. 12. 18. 27.]
 [ 1.  4.  5. 16. 20. 25. 64. 80. 100. 125.]
 [ 1.  6.  7. 36. 42. 49. 216. 252. 294. 343.]]
```

Program Name Loading dataset
demo9.py

```
import pandas as pd

df = pd.read_csv("poly_dataset.csv")

print(df)
```

Output

	Position	Level	Salary
0	Business Analyst	1	45000
1	Junior Consultant	2	50000
2	Senior Consultant	3	60000
3	Manager	4	80000
4	Country Manager	5	110000
5	Region Manager	6	150000
6	Partner	7	200000
7	Senior Partner	8	300000
8	C-level	9	500000
9	CEO	10	1000000

Program Name Data preparation
demo10.py

```
import pandas as pd

df = pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

print(X)
print()
print(y)
```

Output

```
[[ 1]
 [ 2]
 [ 3]
 [ 4]
 [ 5]
 [ 6]
 [ 7]
 [ 8]
 [ 9]
[10]]

[ 45000  50000  60000  80000 110000 150000 200000 300000 500000
1000000]
```

Program Name Plotting the dataset
demo11.py

```
import pandas as pd
import matplotlib.pyplot as plt

df=pd.read_csv("poly_dataset.csv")

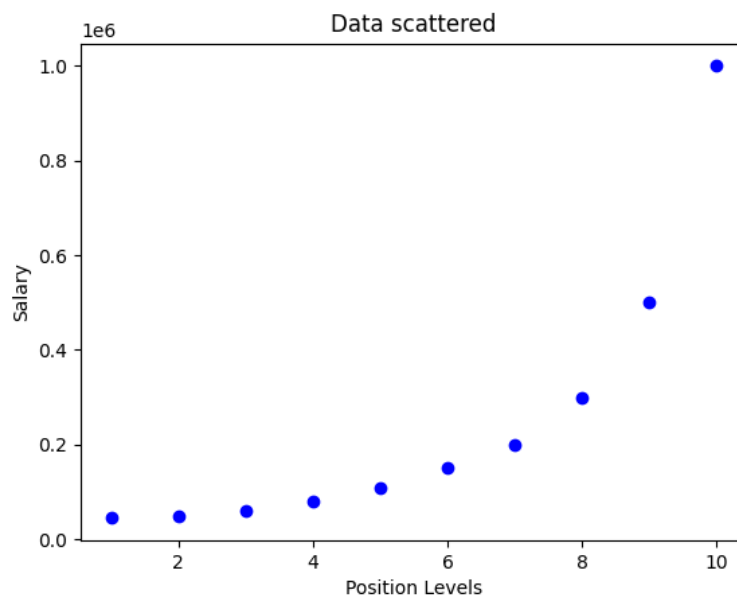
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

plt.scatter(X, y, color="blue")

plt.title("Data scattered")
plt.xlabel("Position Levels")
plt.ylabel("Salary")

plt.show()
```

Output



Program Name Model training
demo12.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Loading the dataset
df = pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

# Model training
lin_regs= LinearRegression()
lin_regs.fit(X, y)

print("Model got trained")
```

Output

Model got trained

Program Name Model prediction
demo13.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Loading the dataset
df = pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

# Model training
lin_regs= LinearRegression()
lin_regs.fit(X, y)

print("Model got trained")
print(lin_regs.predict([[6.5]]) )
```

Output

```
[330378.78787879]
```

Program Name Plotting the dataset
demo14.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Loading the dataset
df=pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

# Model training
lin_regs= LinearRegression()
lin_regs.fit(X, y)

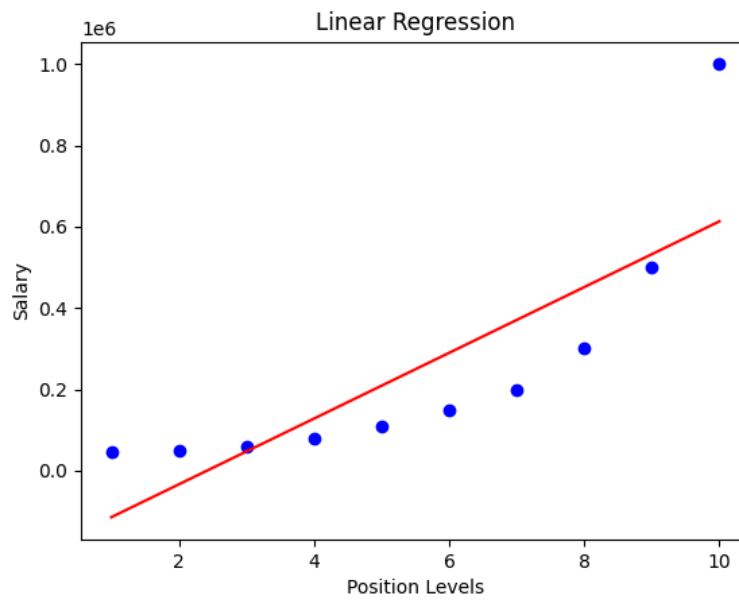
plt.scatter(X, y, color="blue")

plt.plot(X, lin_regs.predict(X), color = "red")

plt.title("Linear Regression")
plt.xlabel("Position Levels")
plt.ylabel("Salary")

plt.show()
```

Output



4. Lets create Polynomial features

- ✓ We need to use PolynomialFeatures class to get polynomial features.

```
Program Name    Fitting the Polynomial regression to the dataset
demo15.py

import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Loading the dataset
df=pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

poly_regs= PolynomialFeatures(degree = 1)

x_poly= poly_regs.fit_transform(X)

print(x_poly)
```

Output

	Position	Level	Salary
0	Business Analyst	1	45000
1	Junior Consultant	2	50000
2	Senior Consultant	3	60000
3	Manager	4	80000
4	Country Manager	5	110000
5	Region Manager	6	150000
6	Partner	7	200000
7	Senior Partner	8	300000
8	C-level	9	500000
9	CEO	10	1000000


```
[[ 1.  1.]  
 [ 1.  2.]  
 [ 1.  3.]  
 [ 1.  4.]  
 [ 1.  5.]  
 [ 1.  6.]  
 [ 1.  7.]  
 [ 1.  8.]  
 [ 1.  9.]  
 [ 1. 10.]
```

Program Name Fitting the Polynomial regression to the dataset
demo16.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Loading the dataset
df=pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

poly_regs= PolynomialFeatures(degree = 2)

x_poly= poly_regs.fit_transform(X)

print(x_poly)
```

Output

	Position	Level	Salary
0	Business Analyst	1	45000
1	Junior Consultant	2	50000
2	Senior Consultant	3	60000
3	Manager	4	80000
4	Country Manager	5	110000
5	Region Manager	6	150000
6	Partner	7	200000
7	Senior Partner	8	300000
8	C-level	9	500000
9	CEO	10	1000000


```
[[ 1.  1.  1.]  
[ 1.  2.  4.]  
[ 1.  3.  9.]  
[ 1.  4. 16.]  
[ 1.  5. 25.]  
[ 1.  6. 36.]  
[ 1.  7. 49.]  
[ 1.  8. 64.]  
[ 1.  9. 81.]  
[ 1. 10. 100.]]
```


Program Name Fitting the Polynomial regression to the dataset
demo17.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Loading the dataset
df=pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

poly_regs= PolynomialFeatures(degree = 2)
x_poly= poly_regs.fit_transform(X)

model = LinearRegression()
model.fit(x_poly, y)

print("Fitting the Polynomial regression to the dataset ")
```

Output

Fitting the Polynomial regression to the dataset

Program Name Plotting Polynomial Regression features
demo18.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt

# Loading the dataset
df=pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

#Fitting the Polynomial regression to the dataset
poly_regs= PolynomialFeatures(degree = 2)
x_poly = poly_regs.fit_transform(X)
model =LinearRegression()
model.fit(x_poly, y)

# Plotting Polynomial Regression

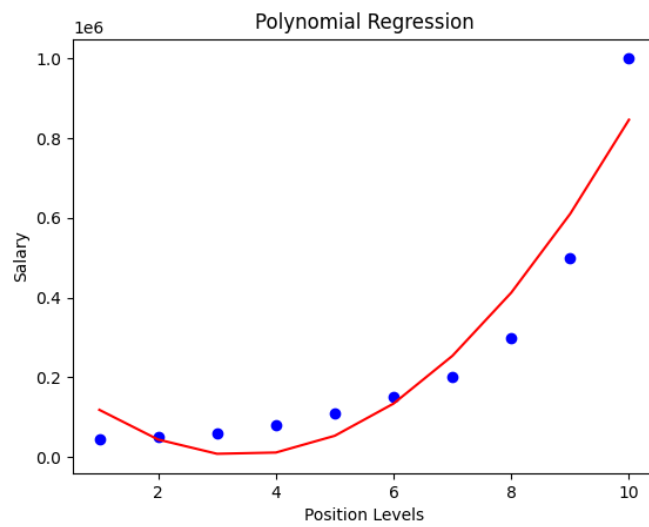
plt.scatter(X, y, color="blue")

plt.plot(X, model.predict(x_poly), color="red")

plt.title("Polynomial Regression")
plt.xlabel("Position Levels")
plt.ylabel("Salary")

plt.show()
```

Output



Program Name Plotting Polynomial Regression features
demo19.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt

# Loading the dataset
df=pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

#Fitting the Polynomial regression to the dataset
poly_regs= PolynomialFeatures(degree = 3)
x_poly = poly_regs.fit_transform(X)
model =LinearRegression()
model.fit(x_poly, y)

# Plotting Polynomial Regression

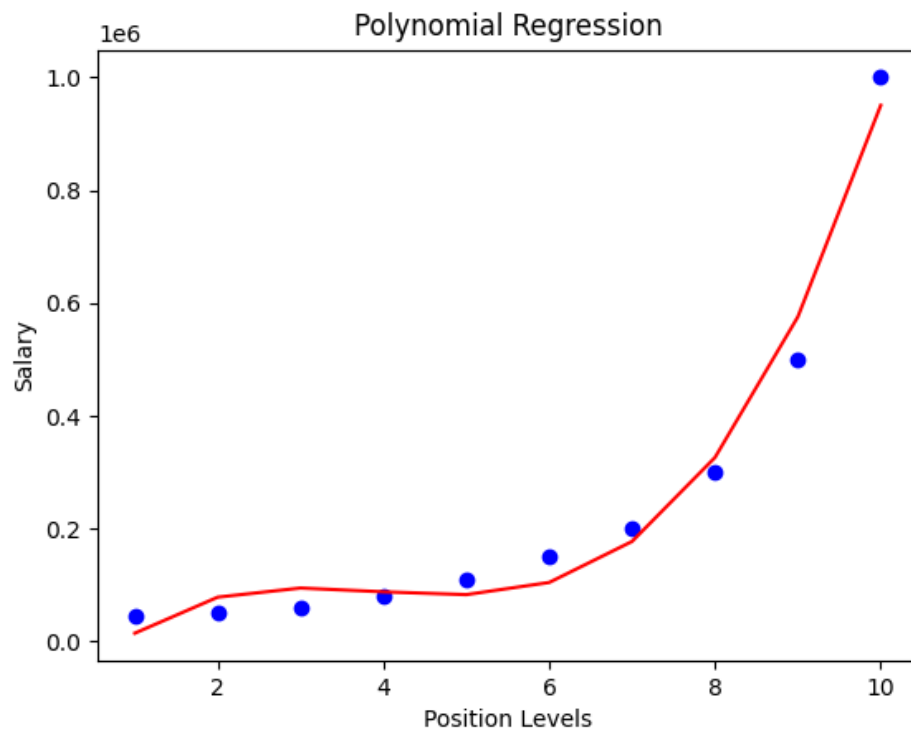
plt.scatter(X, y, color="blue")

plt.plot(X, model.predict(x_poly), color="red")

plt.title("Polynomial Regression")
plt.xlabel("Position Levels")
plt.ylabel("Salary")

plt.show()
```

Output



Program Name Plotting Polynomial Regression features
demo20.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt

# Loading the dataset
df=pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

#Fitting the Polynomial regression to the dataset
poly_regs= PolynomialFeatures(degree = 4)
x_poly = poly_regs.fit_transform(X)
model =LinearRegression()
model.fit(x_poly, y)

# Plotting Polynomial Regression

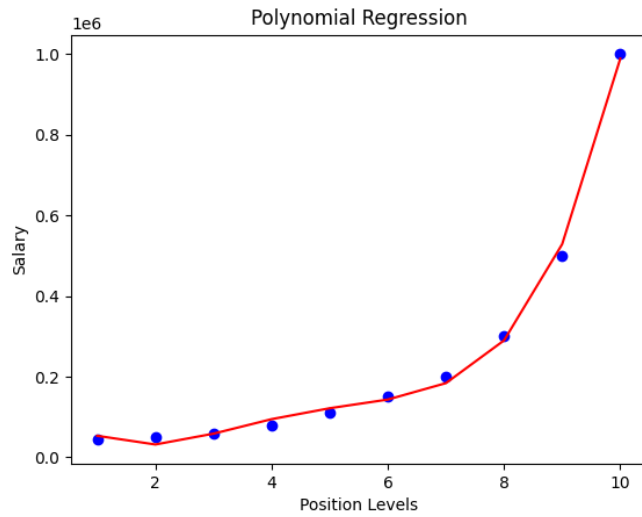
plt.scatter(X, y, color="blue")

plt.plot(X, model.predict(x_poly), color="red")

plt.title("Polynomial Regression")
plt.xlabel("Position Levels")
plt.ylabel("Salary")

plt.show()
```

Output



Program Name Predicting result
demo21.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt

# Loading the dataset
df=pd.read_csv("poly_dataset.csv")

# Data preparation
X = df.iloc[:, 1:2].values
y = df.iloc[:, 2].values

#Fitting the Polynomial regression to the dataset
poly_regs= PolynomialFeatures(degree = 4)
x_poly= poly_regs.fit_transform(X)
model =LinearRegression()
model.fit(x_poly, y)

# Prediction with Polynomial Regression
poly_pred = model.predict(poly_regs.fit_transform([[6.5]]))
print(poly_pred)
```

Output [158862.45265155]

Note

- ✓ LinearRegression predicted output is : [330378.78787879]
- ✓ Polynomial Regression predicted output is : [158862.45265155]

Conclusion

- ✓ Polynomial Regression predicted output is the **accurate** one according to the discussion

11. Data Science – Machine Learning – Multiple Linear Regression

Contents

1. Multiple Linear Regression	2
2. Problem statement	2
3. Dataset	3
4. Machine learning Terminology	4
4.1. Features and label	4
4.2. Models	4
4.3. Prediction	4
4.4. Formula	5

11. Data Science – Machine Learning – Multiple Linear Regression

1. Multiple Linear Regression

- ✓ Multiple Linear Regression explains the relationship between a single dependent continuous variable and more than one independent variable.

2. Problem statement

- ✓ Assuming that we are planning to buy a new house and need to predict the price of a house.
- ✓ Here price **depends** on **area** (square feet), **bed rooms** and **age** of the home (in years).
- ✓ Given these prices we have to **predict prices** of new homes based on area, bed rooms and age
- ✓ Given these home prices find out price of a home that has,
 - 3000 sq ft area, 3 bedrooms, 40 year old
 - 2500 sq ft area, 4 bedrooms, 5 year old

3. Dataset

- ✓ homeprices1.csv is dataset we are using in this example
- ✓ This dataset contains columns as,
 - Area
 - Bedrooms
 - Age
 - Price

Area	Bedrooms	Age	price
2600	3	20	550000
3000	4	15	565000
3200		18	610000
3600	3	30	595000
4000	5	8	760000
4100	6	8	810000

4. Machine learning Terminology

4.1. Features and label

- ✓ Here **area**, **bedrooms**, **age** are called independent variables or features whereas price is a dependant variable

4.2. Models

- ✓ A machine learning model is simply a rule, or a formula, which predicts a label from the features.
- ✓ In this case, the model is the equation we found for the price.

4.3. Prediction

- ✓ The prediction is simply the output of the model.

4.4. Formula

$$y = m_1x_1 + m_2x_2 + m_3x_3 + b$$

$$price = m_1 * area + m_2 * bedrooms + m_3 * age + b$$

Dependent variable

Independent variables (**features**)

$$price = m_1 * area + m_2 * bedrooms + m_3 * age + b$$

Coefficients

The diagram shows the formula $price = m_1 * area + m_2 * bedrooms + m_3 * age + b$. A red arrow points from the text 'Dependent variable' to the word 'price'. Three red arrows point from the text 'Independent variables (features)' to the words 'area', 'bedrooms', and 'age'. Three purple arrows point from the text 'Coefficients' to the terms ' m_1 ', ' m_2 ', and ' m_3 '.

$$y = m_1x_1 + m_2x_2 + m_3x_3 + b$$

Program Name Loading house prices dataset
demo1.py

```
import pandas as pd

# Loading the dataset

df = pd.read_csv("homeprices1.csv")

print(df)
```

Output

	area	bedrooms	age	price
0	2600	3.0	20	550000
1	3000	4.0	15	565000
2	3200	NaN	18	610000
3	3600	3.0	30	595000
4	4000	5.0	8	760000
5	4100	6.0	8	810000

Program Name Data pre-processing – Finding mean of bedrooms column
demo2.py

```
import pandas as pd

df = pd.read_csv("homeprices1.csv")

# Mean of the bedrooms
print("Mean of the bedrooms")
print(df.bedrooms.median())
```

Output

```
Mean of the bedrooms
4.0
```


Program Name Data pre-processing - Fill NA values with median value of a column
demo3.py

```
import pandas as pd

df = pd.read_csv("homeprices1.csv")

# Data loading
print("Filling missing value with mean\n")

# Data preprocessing

m = df.bedrooms.median()
df.bedrooms = df.bedrooms.fillna(m)
print(df)
```

Output

```
Filling missing value with mean
   area  bedrooms  age  price
0  2600         3.0  20  55000
1  3000         4.0  15  56500
2  3200         4.0  18  61000
3  3600         3.0  30  59500
4  4000         5.0   8  76000
5  4100         6.0   8  81000
```

Program Name Model training
demo4.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Data loading
df = pd.read_csv("homeprices1.csv")

# Data preprocessing
m = df.bedrooms.median()
df.bedrooms = df.bedrooms.fillna(m)
a = df.drop('price', axis = 'columns')

# Model training
reg = LinearRegression()
reg.fit(a.values, df.price)
print("Model trained")
```

Output

Model trained

Program Name Finding intercept
demo5.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Data loading
df=pd.read_csv("homeprices1.csv")

# Data preprocessing
m = df.bedrooms.median()
df.bedrooms = df.bedrooms.fillna(m)
a = df.drop('price', axis = 'columns')

# Model training
reg = LinearRegression()
reg.fit(a.values, df.price)

print("Intercept is:")
print(reg.intercept_)
```

Output

```
Intercept is:
221323.00186540408
```

Program Name Finding coefficients
demo6.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Data loading
df = pd.read_csv("homeprices1.csv")

# Data preprocessing
m = df.bedrooms.median()
df.bedrooms = df.bedrooms.fillna(m)
a = df.drop('price', axis = 'columns')

# Model training
reg = LinearRegression()
reg.fit(a.values, df.price)

print("Coefficients are:")
print(reg.coef_)
```

Output

```
Coefficients are:
[ 112.06244194 23388.88007794 -3231.71790863]
```

Program Name price of home with 3000 sqr ft area, 3 bedrooms, 40 year old demo7.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Data loading
df = pd.read_csv("homeprices1.csv")

# Data preprocessing
m = df.bedrooms.median()
df.bedrooms = df.bedrooms.fillna(m)
a = df.drop('price', axis='columns')

# Model training
reg = LinearRegression()
reg.fit(a.values, df.price)

# Prediction
print("price of home with 3000 sqr ft area, 3 bedrooms, 40 year old")
print(reg.predict([[3000, 3, 40]]))
```

Output

```
price of home with 3000 sqr ft area, 3 bedrooms, 40 year old
[498408.25158031]
```

Program Name price of home with 3000 sqr ft area, 3 bedrooms, 40 year old demo8.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Data loading
df = pd.read_csv("homeprices1.csv")

# Data preprocessing
m = df.bedrooms.median()
df.bedrooms = df.bedrooms.fillna(m)
a = df.drop('price', axis = 'columns')

# Model training
reg = LinearRegression()
reg.fit(a.values, df.price)

# Prediction
print("price of home with 3000 sqr ft area, 3 bedrooms, 40 year old")

b = 112.06244194*3000 + 23388.88007794*3 + -
3231.71790863*40 + 221323.00186540384
print(b)
```

Output

```
price of home with 3000 sqr ft area, 3 bedrooms, 40 year old
[498408.25158031]
```

Program Name price of home with 2500 sqr ft area, 4 bedrooms, 5 year old demo9.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression

# Data loading
df = pd.read_csv("homeprices1.csv")

# Data preprocessing
m = df.bedrooms.median()
df.bedrooms = df.bedrooms.fillna(m)
a = df.drop('price', axis = 'columns')

# Model training
reg = LinearRegression()
reg.fit(a.values, df.price)

# Prediction
print("price of home with 2500 sqr ft area, 4 bedrooms, 5 year old")

print(reg.predict([[2500, 4, 5]]))
```

Output

```
price of home with 2500 sqr ft area, 4 bedrooms, 5 year old
[578876.03748933]
```

12. Data Science – Machine Learning – Pickling and Unpickling

Contents

1. Pickling.....	2
2. Unpickling	2

12. Data Science – Machine Learning – Pickling and Unpickling

- ✓ Based on requirement we can write total state of object to the file and we can read total object information from the file.

1. Pickling

- ✓ The process of writing state of object to the file is called as **pickling**.
- ✓ We can implement pickling and unpickling by using python **pickle module**
- ✓ We can do pickling by using `dump(object, file)` predefined function in pickle module

- `pickle.dump(object, file)`

2. Unpickling

- ✓ The process of reading state of an object from the file is called **unpickling**.
- ✓ We can do unpickling by using `load(file)` predefined function in pickle module

- `obj = pickle.load(file)`

Program Name Pickling and unpickling example
demo1.py

```
import pickle

class Employee:
    def __init__(self, eno, ename, esal):
        self.eno = eno
        self.ename = ename
        self.esal = esal

    def display(self):
        print("Number is:", self.eno)
        print("Name is:", self.ename)
        print("Salary is:", self.esal)

with open("emp.dat", "wb") as f:
    e = Employee(100, "Daniel", 1000)
    pickle.dump(e, f)
    print("Pickling of Employee Object completed...\n")

with open("emp.dat", "rb") as f:
    obj = pickle.load(f)
    print("Printing Employee Information after unpickling")
    obj.display()
```

Output

Pickling of Employee Object completed...

Printing Employee Information after unpickling

Number is: 100

Name is: Daniel

Salary is: 1000

13. Data Science – Machine Learning – Saving model, Joblib & Pickling

Contents

1. Save model	2
2. Pickling.....	3

13. Data Science – Machine Learning – Saving model, Joblib & Pickling

1. Save model

- ✓ Once the model got created then we can save that model.
- ✓ There are two ways to save the model
 - By using python File IO pickle concept
 - By using Joblib library

Program Name	Predict price of a home with area = 5000 sq ft demo1.py
	<pre>import pandas as pd from sklearn.linear_model import LinearRegression df = pd.read_csv("homeprices.csv") new_df = df.drop('price', axis='columns') reg = LinearRegression() reg.fit(new_df, df.price) # Predictions print(reg.predict([[5000]]))</pre>
Output	[859554.79452055]

2. Pickling

- ✓ The process of writing state of object to the file is called as **pickling**.

Program Name	Save the model into pickle file demo2.py
	<pre>import pandas as pd from sklearn.linear_model import LinearRegression import pickle df = pd.read_csv("homeprices.csv") new_df = df.drop('price', axis='columns') model = LinearRegression() model.fit(new_df.values, df.price.values) with open('model_pickle', 'wb') as file: pickle.dump(model, file) with open('model_pickle', 'rb') as file: model1 = pickle.load(file) print(model1.predict([[5000]]))</pre>
Output	[859554.79452055]

Program Name Doing prediction by using saved model
demo3.py

```
import pickle
```

```
with open('model_pickle', 'rb') as file:  
    model1 = pickle.load(file)  
    print(model1.predict([[5000]]))
```

Output

```
[859554.79452055]
```

Program Name Doing prediction by using saved model
demo4.py

```
import pickle
```

```
with open('model_pickle', 'rb') as file:  
    model1 = pickle.load(file)  
    print(model1.predict([[6000]]))
```

Output

```
[995342.46575342]
```

Program Name Save Trained Model Using Joblib
demo5.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression
import joblib

df = pd.read_csv("homeprices.csv")
new_df = df.drop('price', axis='columns')

model = LinearRegression()
model.fit(new_df.values, df.price.values)

joblib.dump(model, 'model_joblib')

print("Model got saved")
```

Output

Model got saved

Program Do prediction with saved model
Name demo6.py

```
import joblib

mj = joblib.load('model_joblib')

print(mj.predict([[5000]]))
```

Output

[859554.79452055]

14. Data Science – Machine Learning – Cost function

Contents

1. Cost Functions in Machine Learning 2

2. Why Cost Functions in Machine Learning 2

3. How cost function works?..... 2

4. Goal of training..... 2

5. Useful sources 3

14. Data Science – Machine Learning – Cost function

1. Cost Functions in Machine Learning

- ✓ Cost functions are also called as Loss functions in machine learning.
- ✓ These are **optimization** techniques.
- ✓ There are many cost functions in machine learning
- ✓ There are different cost functions for regression and classification.

2. Why Cost Functions in Machine Learning

- ✓ During the training phase, the model assumes the initial weights randomly and tries to make a prediction on training data.
- ✓ So here, how the model gets to know how much "far" it was from the prediction?
- ✓ Model needs this information so that it can adjust the weight accordingly (using gradient descent) in the next iteration on training data.

3. How cost function works?

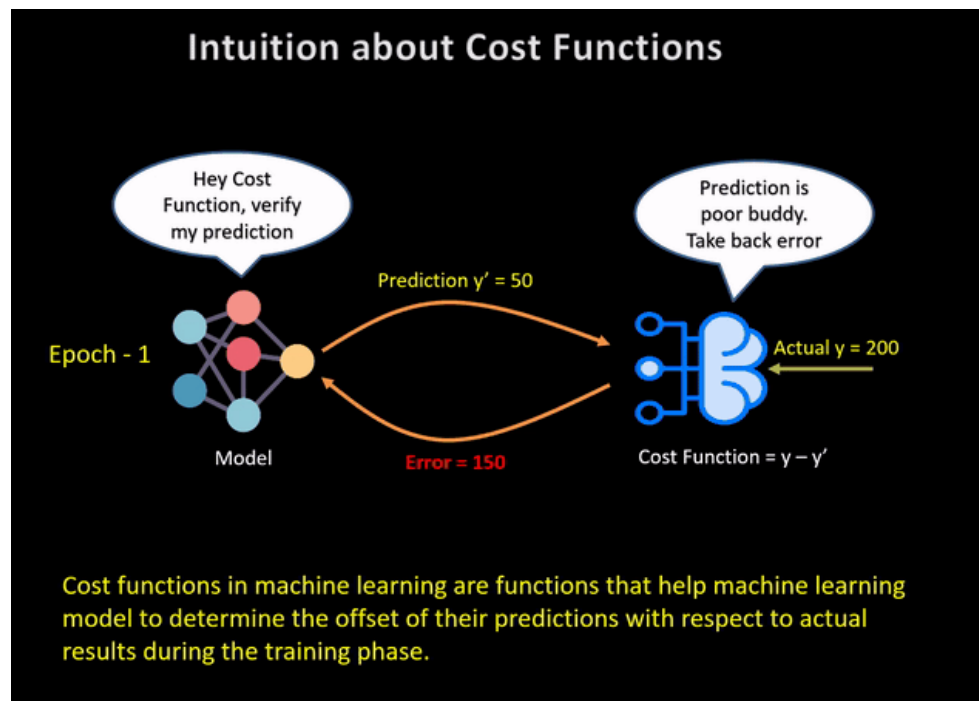
- ✓ Cost function is a function that takes both **predicted outputs** and **actual outputs** from model.
- ✓ Then it calculates how much error in the model to predict the good results.
- ✓ In next step the model tries to adjust this error for the next iteration on training data to reach optimization level

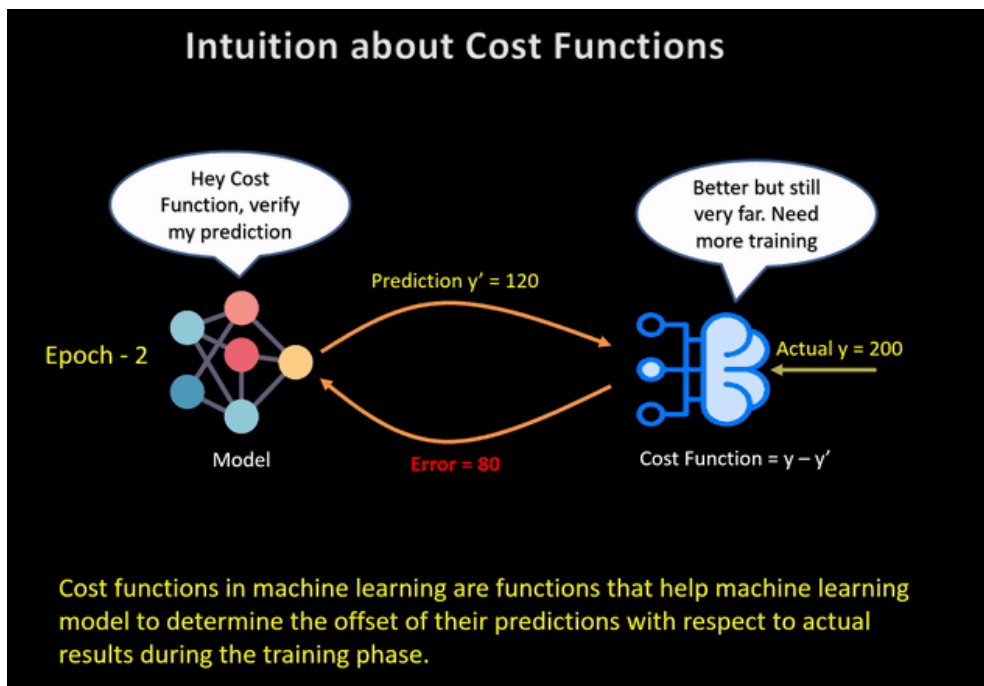
4. Goal of training

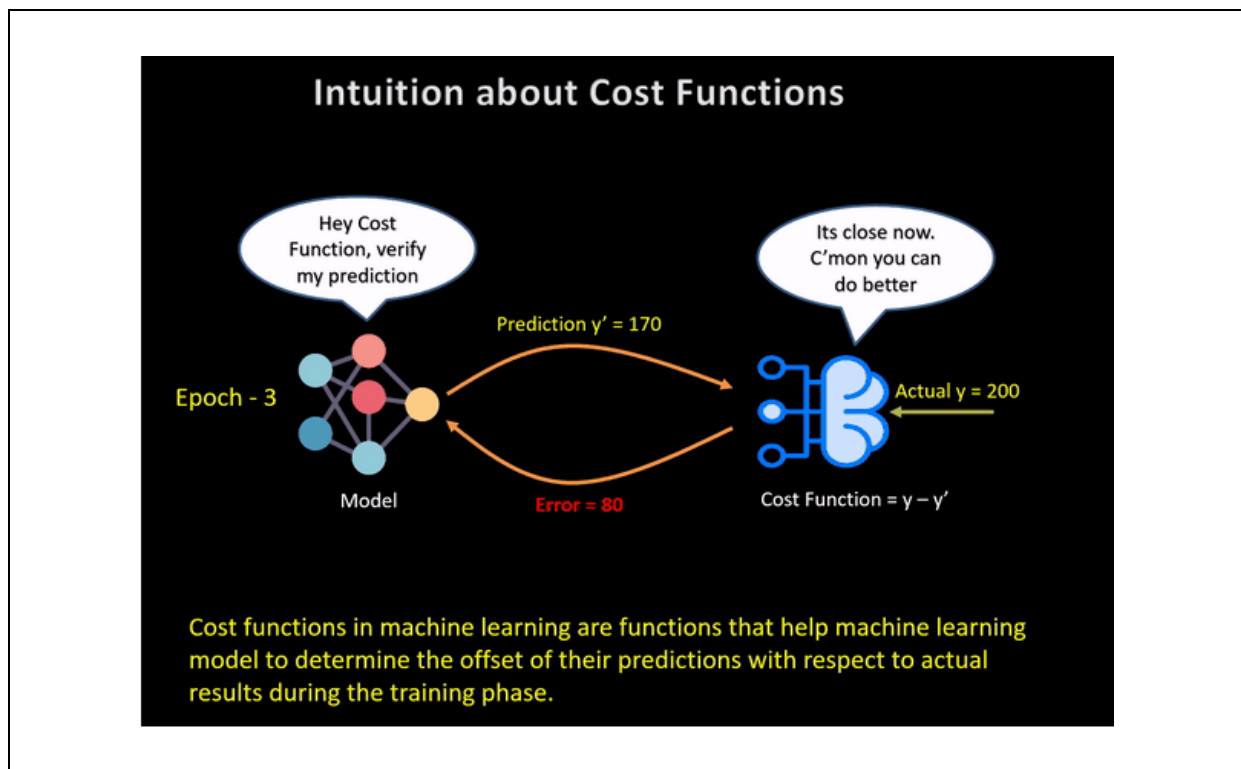
- ✓ The goal of the training phase is to come up with weights that minimize the **error in every iteration**.
- ✓ This way of training is required in machine learning to optimize model

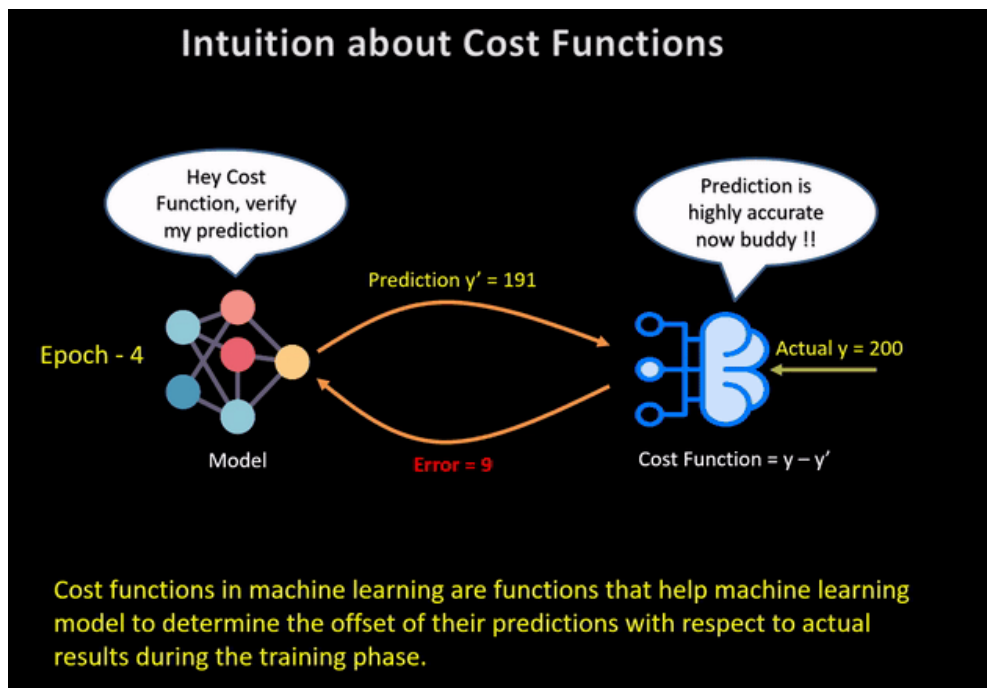
5. Useful sources

- ✓ These images are highly recommended to practice









15. Data Science – Machine Learning – Regression Cost functions

Contents

1. Cost Functions for Regression.....	2
2. Types of regression metrics	2
3. Distance Based Error	3
4. Mean Squared Error	4
5. Root Mean Squared Error	6
6. Mean Absolute Error	7
7. Formulas	8

15. Data Science – Machine Learning – Regression Cost functions

1. Cost Functions for Regression

- ✓ In regression, the model predicts an output value for each training data during the training phase.
- ✓ The cost functions for regression are calculated on **distance-based error**.

2. Types of regression metrics

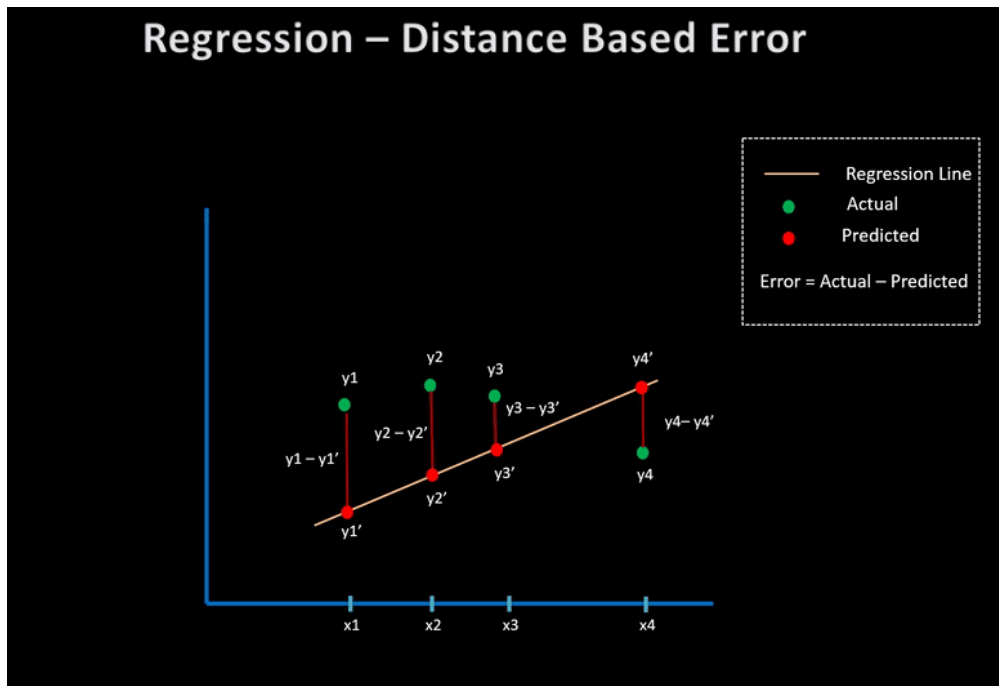
- ✓ There are three metrics in regression,
 - Mean Squared Error (MSE).
 - Root Mean Squared Error (RMSE)
 - Mean Absolute Error (MAE)

3. Distance Based Error

- ✓ Assuming that for a given set of input data, the **actual** output was **y** and our regression model **predicts y'** then the error in prediction is calculated as

$$\blacksquare \text{ Error} = y - y'$$

- ✓ This also known as **distance-based error** and it forms the basis of cost functions that are used in regression models.



4. Mean Squared Error

- ✓ The MSE is calculated as the **mean of the squared differences between predicted and expected target values** in a dataset.
- ✓ This value never be negative, since we are always squaring the errors.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Program Means Squared Error
Name demo1.py

```
from sklearn.metrics import mean_squared_error

expected = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
predicted = [1.0, 0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1, 0.0]

mse_value = mean_squared_error(expected, predicted)

print(mse_value)
```

Output

```
0.35000000000000003
```

5. Root Mean Squared Error

- ✓ The Root Mean Squared Error, or RMSE, is an extension of the mean squared error.
 - $RMSE = \sqrt{MSE}$

Program Name	Root Means Squared Error demo2.py
Output	<pre>from sklearn.metrics import mean_squared_error expected = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0] predicted = [1.0, 0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1, 0.0] rmse_value = mean_squared_error(expected, predicted, squared = False) print(rmse_value)</pre> 0.5916079783099616

6. Mean Absolute Error

- ✓ MAE, average absolute difference between predicted and actual values.

Program Name Means Absolute Error
demo3.py

```
from sklearn.metrics import mean_absolute_error

expected = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
predicted = [1.0, 0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1, 0.0]

mae_value = mean_absolute_error(expected, predicted)

print(mae_value)
```

Output

0.5

7. Formulas

Mean squared error	$\text{MSE} = \frac{1}{n} \sum_{t=1}^n e_t^2$
Root mean squared error	$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n e_t^2}$
Mean absolute error	$\text{MAE} = \frac{1}{n} \sum_{t=1}^n e_t $

16. Data Science – Machine Learning – Gradient Descent

Contents

1. Gradient Descent.....	2
2. How Gradient Descent works	2
3. Convergence	3
4. Process behind the gradient	3
5. Steps	5
6. Follow below images.....	7
7. Fixed steps.....	11
8. Learning rate: Reaching minimum error	12

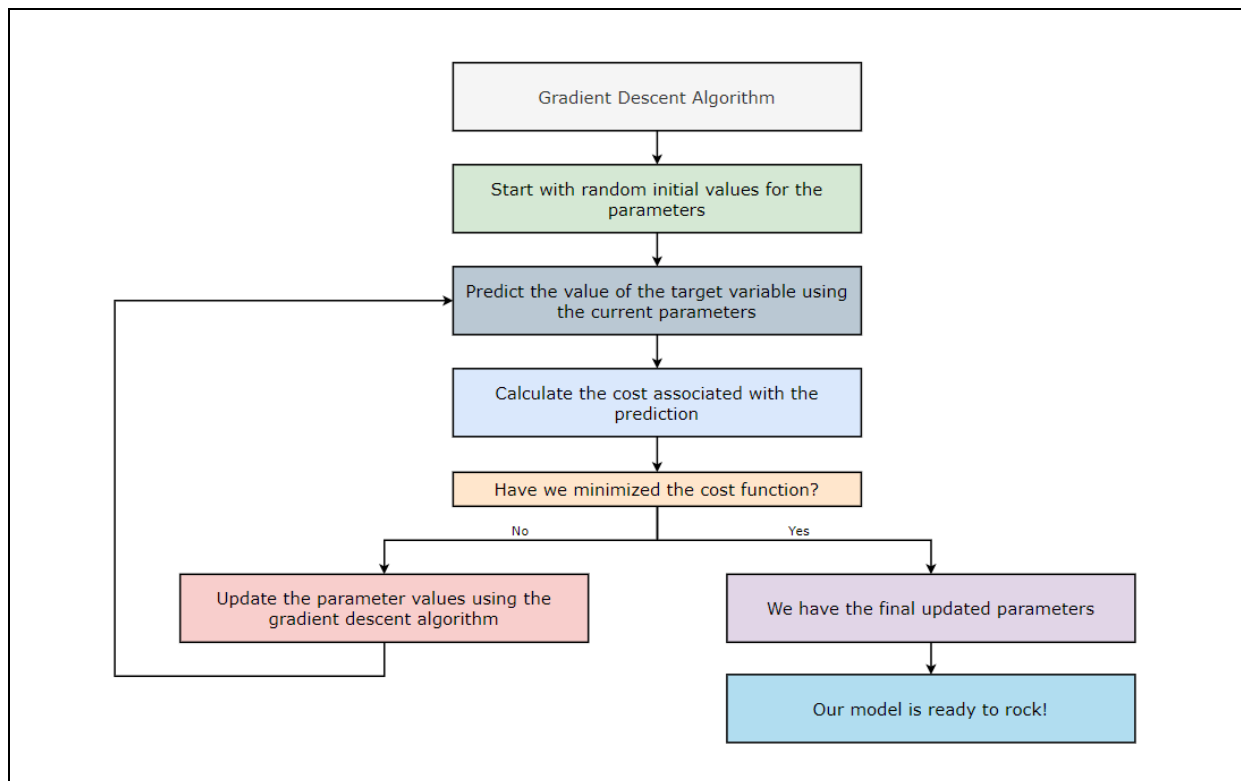
16. Data Science – Machine Learning – Gradient Descent

1. Gradient Descent

- ✓ Gradient descent is an optimization algorithm
- ✓ This algorithm find the values of parameters (coefficients) of a function (f) that minimizes a cost function (cost).

2. How Gradient Descent works

- ✓ 1. Start with random initial values for the parameters.
- ✓ 2. Predict the value of the target variable using the current parameters.
- ✓ 3. Calculate the cost associated with the prediction.
- ✓ 4. Is cost minimized?
 - If yes, then go to step no - 6.
 - If no, then go to step no - 5.
- ✓ 5. Update the parameter values using the gradient descent algorithm and return to step no - 2
- ✓ 6. We have our final updated parameters.
- ✓ 7. Our model is ready.

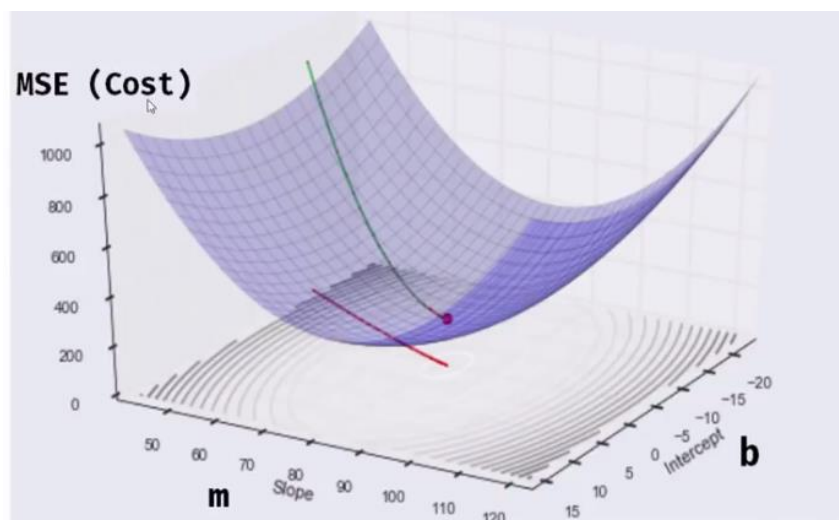


3. Convergence

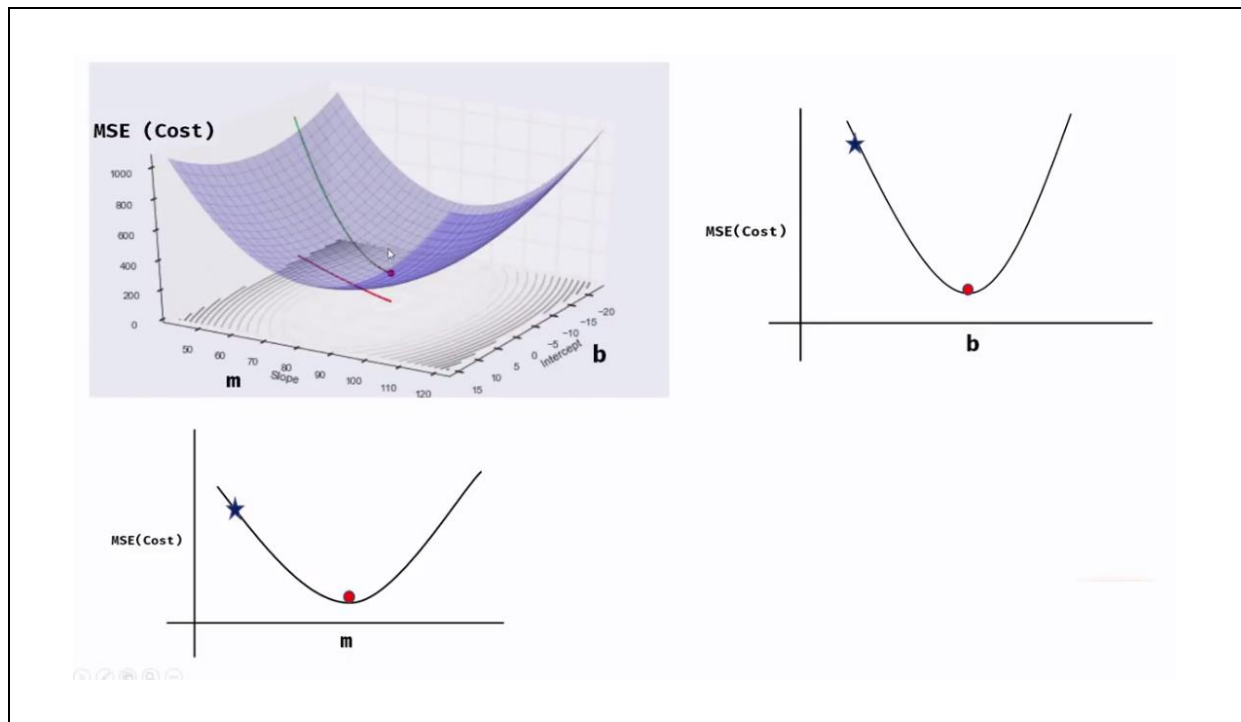
- ✓ Reaching a point in which gradient descent makes very small changes in your objective function is called convergence.
- ✓ It doesn't mean that, it has reached the optimal result but it is somewhat near to that.

4. Process behind the gradient

- ✓ Gradient descent is the algorithm that finds best fit line for given training data set.
- ✓ If we calculate for every value of m and b we will get **MSE**
- ✓ If we plot m and b against MSE then we will get below image

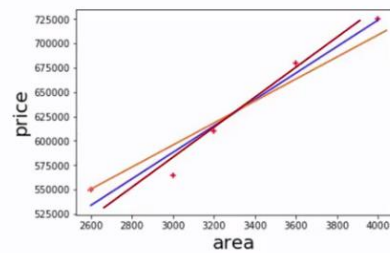
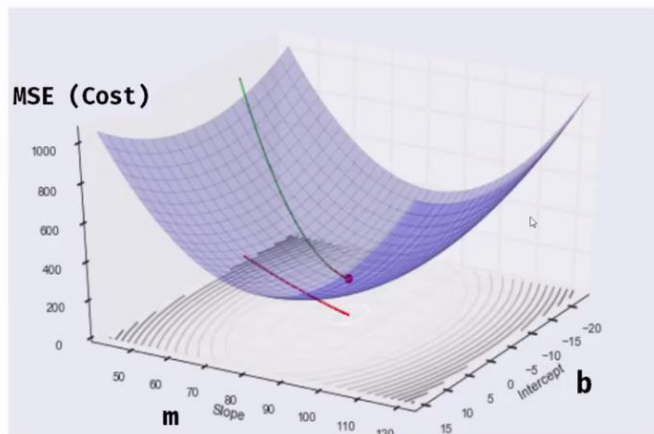


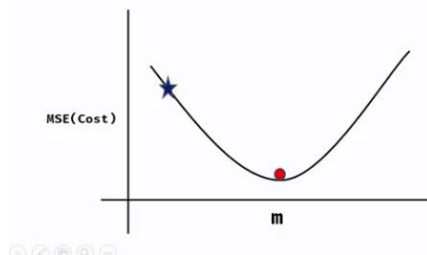
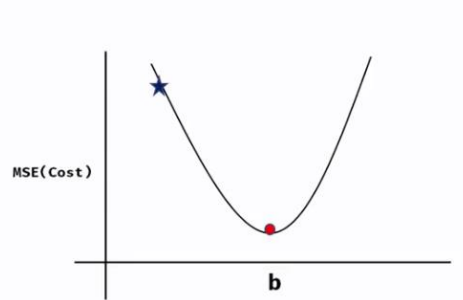
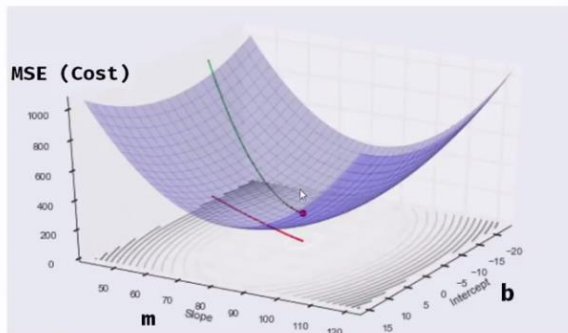
- ✓ Values
 - MSE values 0, 200, 400, 600, 800, 1000 etc
 - m values 50, 60, 70, 80, 90, 100, 110, 120 etc
 - b values 15, 10, 5, 0, -5, -10, -15 etc
- ✓ Here we will get an image which is like a bowl.
- ✓ We need to start some value for m and b , usually let's start from **zero** value



5. Steps

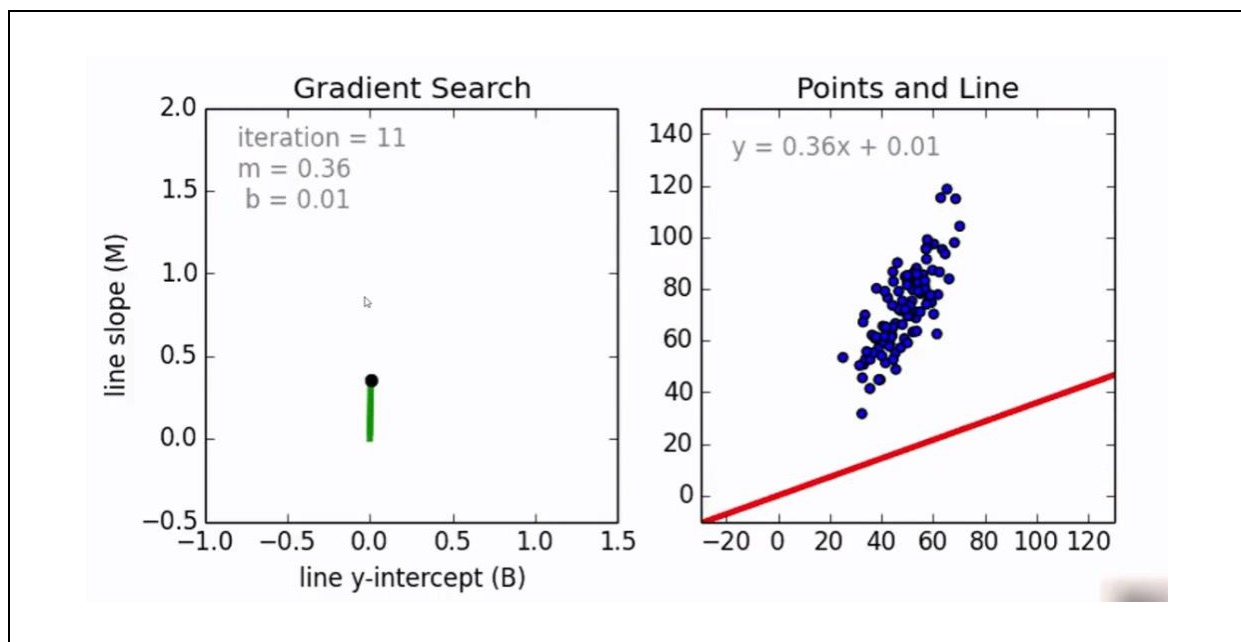
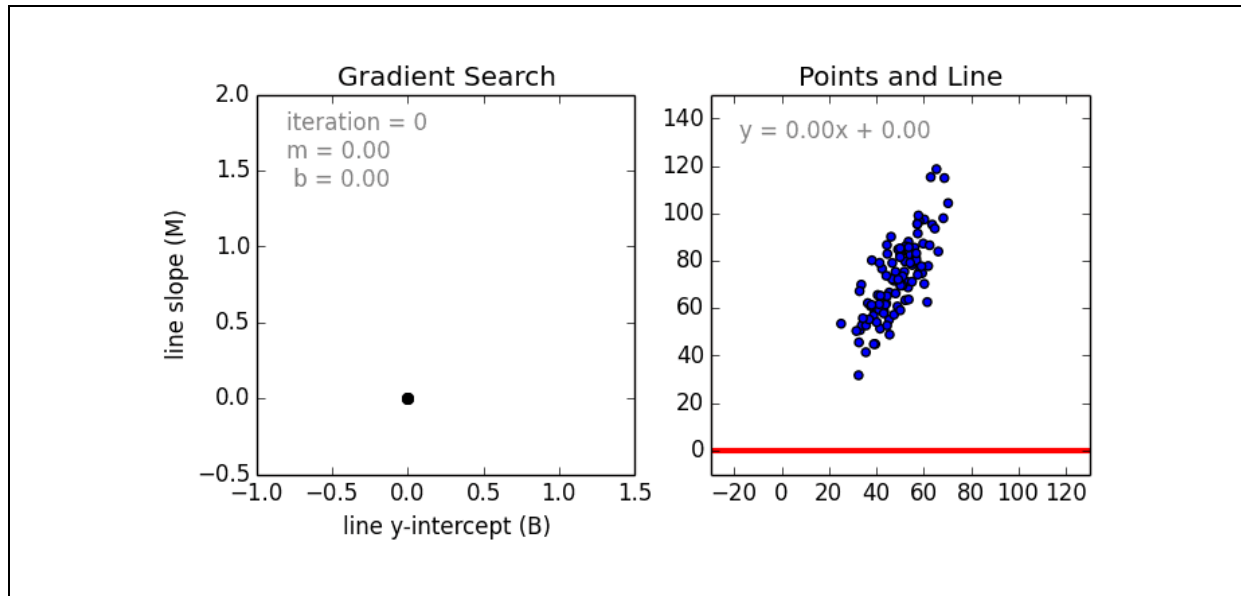
- ✓ Initially lets starts the point with $m = 0$ and $b = 0$
- ✓ Assuming that according to the diagram mse value is 1000 at m , b equals to zero
- ✓ Now just tune the values of m and b with some amount
- ✓ Now check mse value and the error will get reduce
- ✓ So, we need to keep on doing this process until to reach error level to minima(minimum)
- ✓ Once it reach the minima then we got our answer
- ✓ Here we need to use that m and b values, in the prediction function

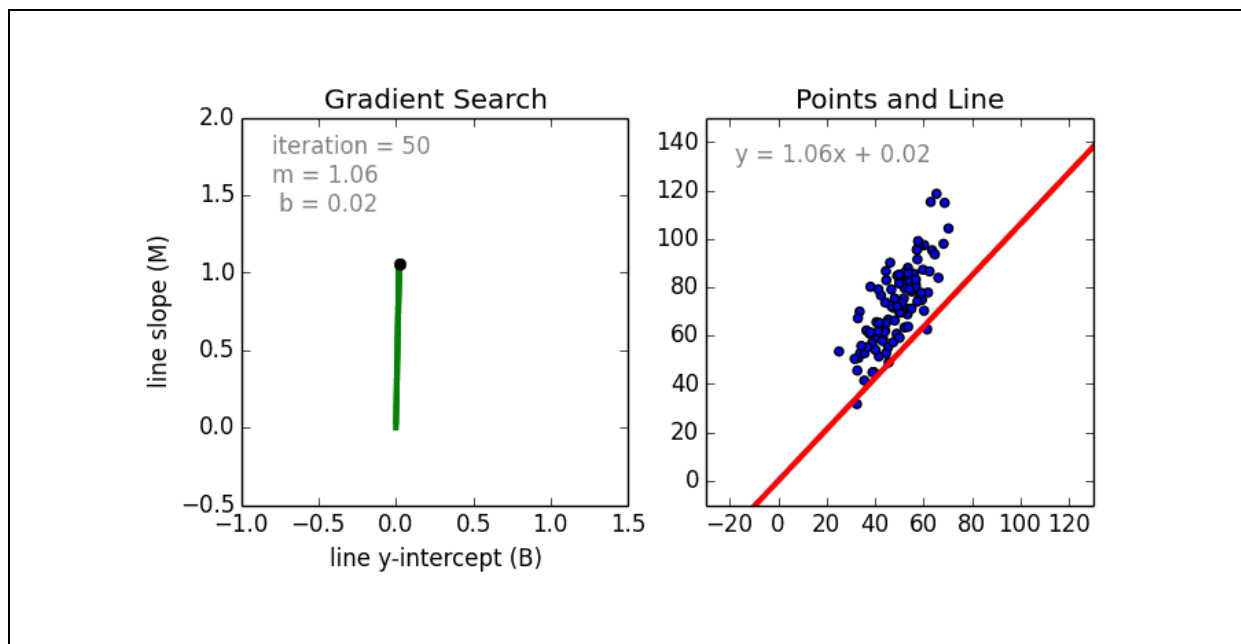
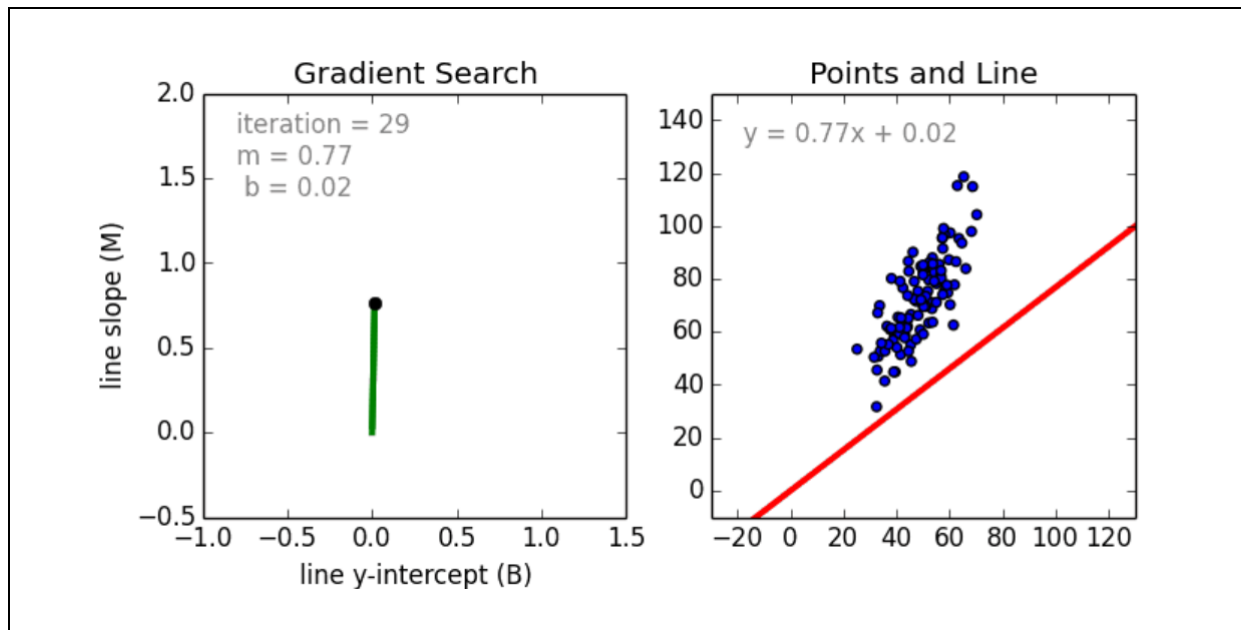


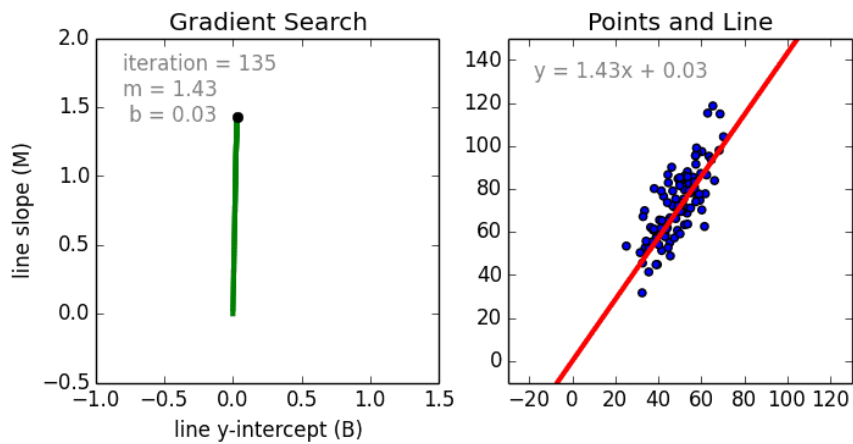
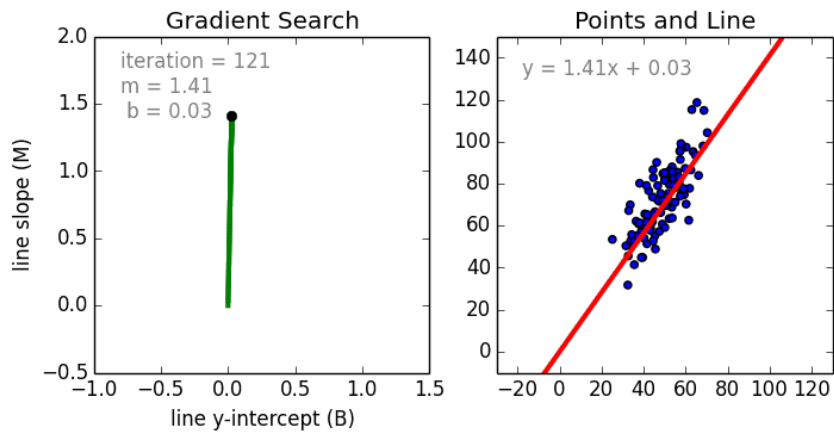


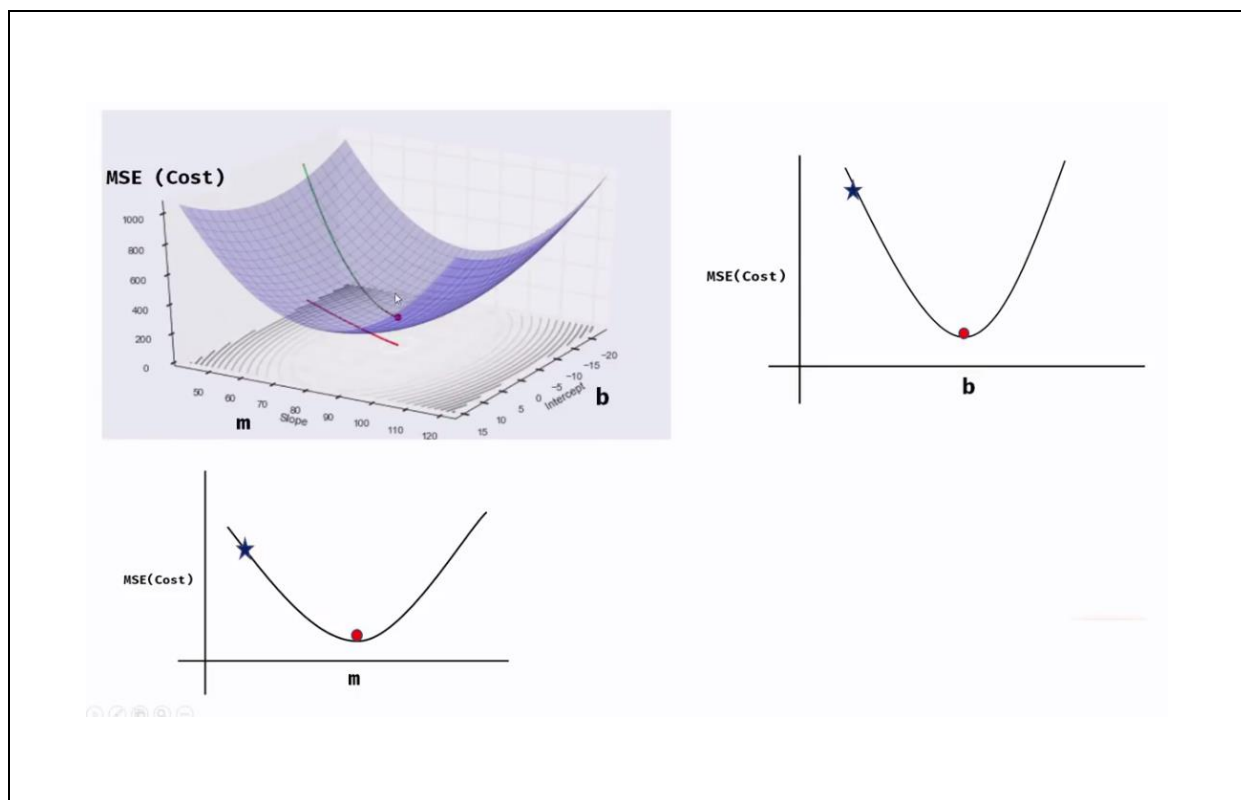
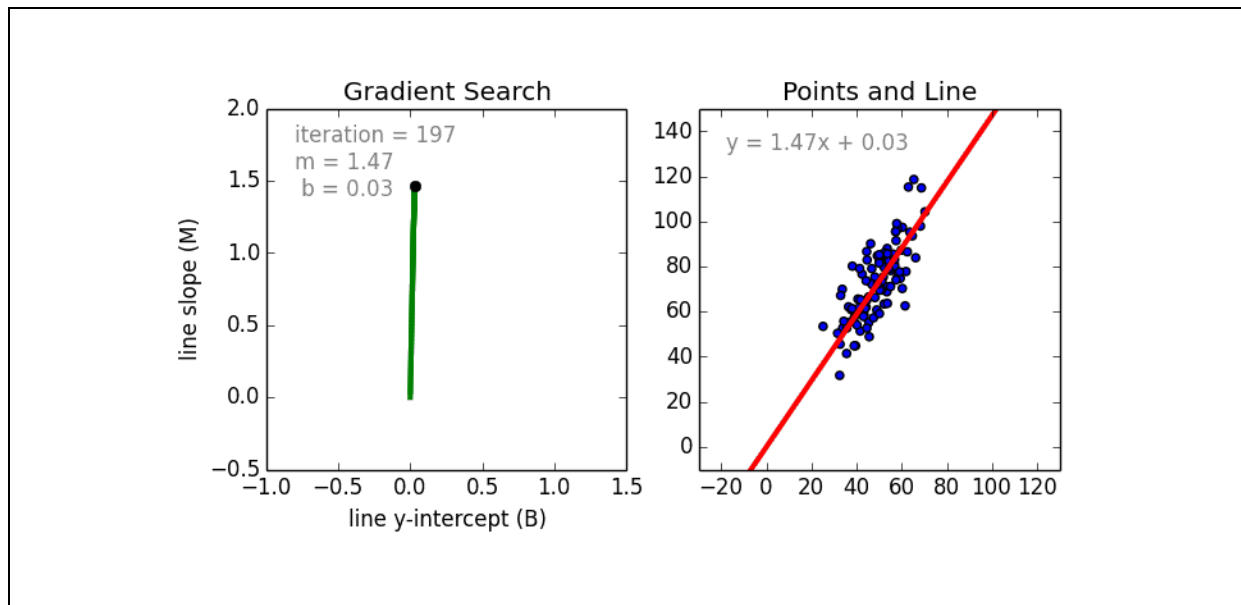
6. Follow below images

✓ Just try to understand below images for better understanding



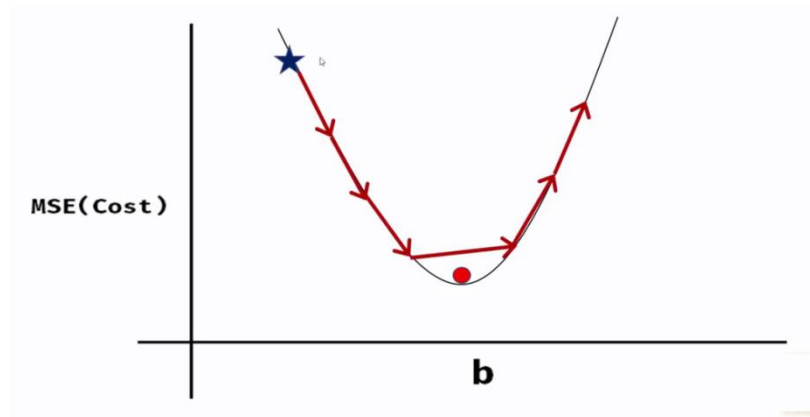






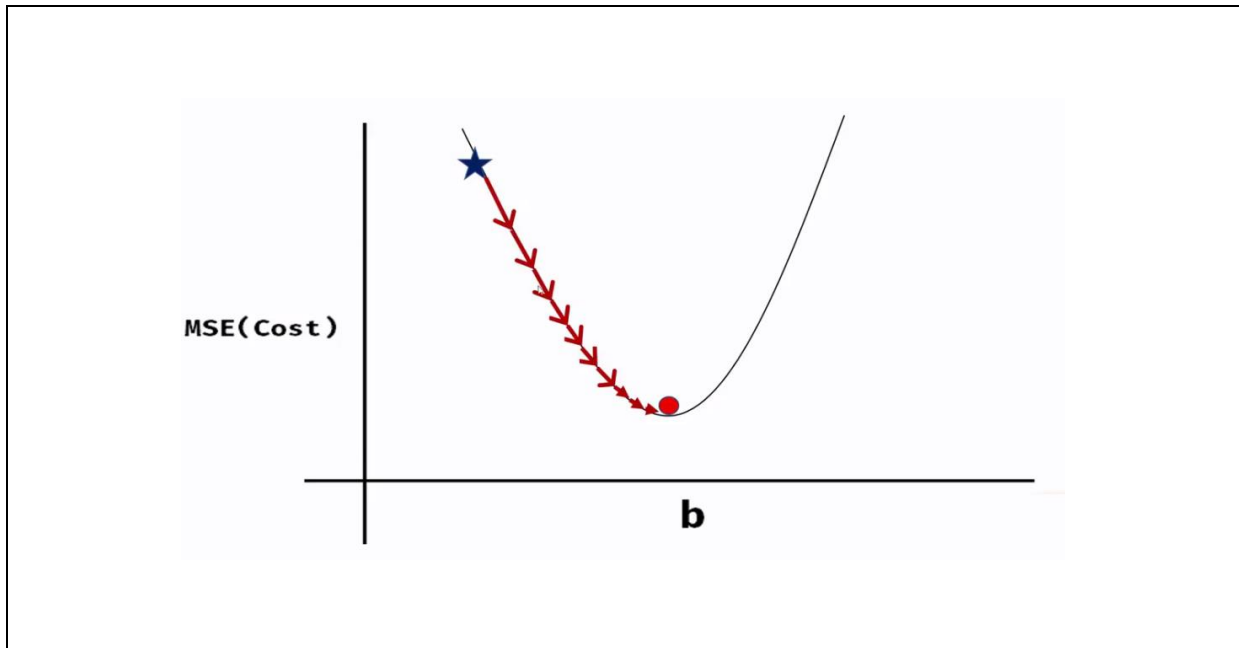
7. Fixed steps

- ✓ If we follow the fixed steps then we will get the below image
- ✓ By following this approach we may miss the global minima value.
- ✓ So, this approach is not going to work in well

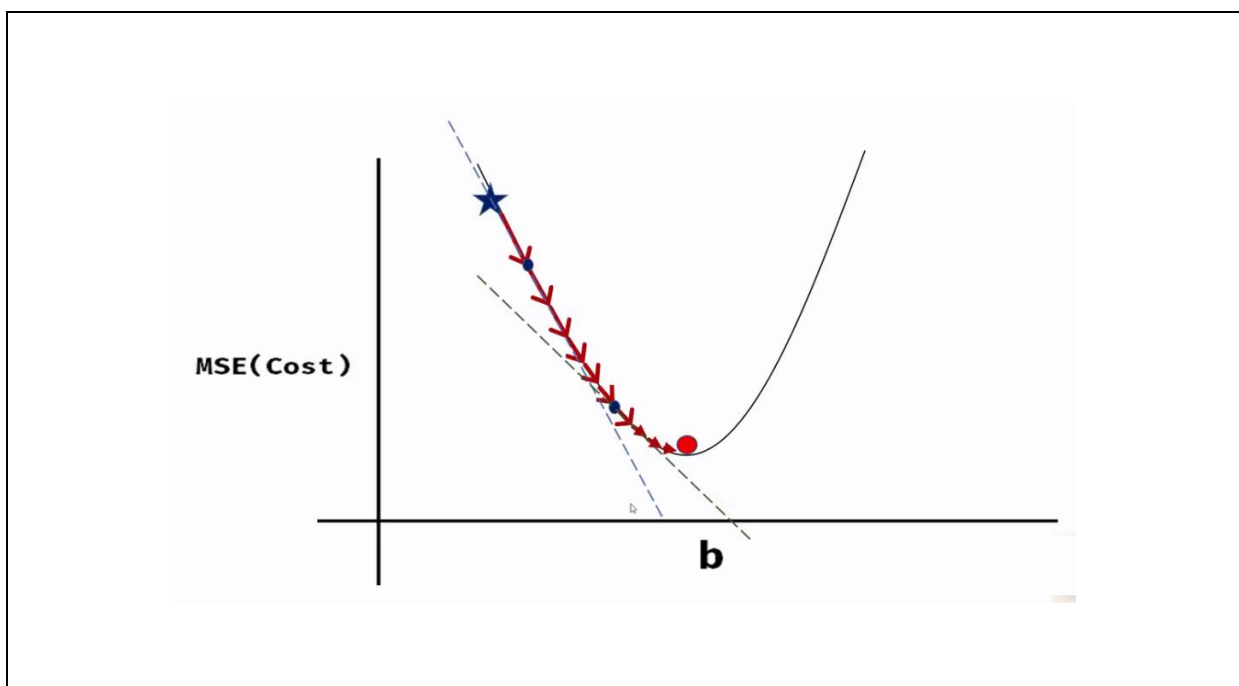


8. Learning rate: Reaching minimum error

- ✓ This approach seems to be good to catch minima value.
- ✓ The learning rate is a tuning parameter in an optimization algorithm.
- ✓ This determines the step size at each iteration while moving toward a minimum of a loss function.



- ✓ At each point if we calculate the slope then we can reach the minimum value



9. Small and large learning rates

- ✓ A small learning rate may lead to the model to take some time to learn
- ✓ A large learning rate will make the model converge as our pointer will shoot and we'll not be able to get to minima.

